Generative AI inference architecture and best practices on AWS

AWS Prescriptive Guidance



Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

AWS Prescriptive Guidance: Generative AI inference architecture and best practices on AWS

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Introduction **1**
Intended audience 1
Objectives 2
Attachments 2
AWS inference stack **3**
Serverless inference layer 3
Managed inference layer 4
Self-managed inference layer 4
Components of an AI inference stack **5**
End user application 5
Inference API server or frontend 6
Inference backend 6
Challenges of inference compared to training **7**
How AI Inference works **9**
The transformer architecture 9
Core components 9
Encoder, decoder, and hybrid architectures 10
Model families and inference patterns 12
LLM inference: Prefill and decode 13
The prefill phase (context encoding) 13
Optional: Vision encoding phase 15
How VLM inference works 15
Why the KV cache exists 15
How it works 16
Selecting an AWS service for AI inference **17**
Amazon Bedrock 17
Amazon Bedrock 17
Amazon SageMaker AI inference endpoints 27
Infrastructure management 27
Pricing model 27
Model architecture support 27
Automatic scaling 28
Inference engine choice 28
Inference optimizations and configurations 28

Supported clients and protocols 29
Amazon SageMaker HyperPod 29
Amazon SageMaker HyperPod 29
Self-managed inference 32
Infrastructure management 32
Training compute reuse 33
Pricing model 33
Model architecture support 33
Automatic scaling 33
Inference engine choice 34
Inference configuration 34
Supported clients and protocols 34
Model optimization techniques 35
Model Compression 35
Quantization (Precision Reduction) 35
Precision hierarchy 35
KV Cache Quantization 36
Model compilation 36
Execution Modes: Eager vs. Graph 37
Kernel Optimization During Compilation 37
Trainium Compilation with Neuron SDK 38
Speculative decoding 38
Model-less Speculation 39
Draft Model Speculation 39
Decoding Head Speculation 41
Multi-LoRA Serving 43
How Multi-LoRA Serving Works 43
Memory and Performance Impact 44
When to Use Multi-LoRA Serving 44
Configuration Examples 44
Applying inference optimizations in SageMaker AI 45
System-level optimizations 46
Inference Engine Tuning 46
Anatomy of Inference Requests 46
Core Parameters and Their Implications 47
Enabling Chunked Prefill 48

Parallelism 49
Parallelism Strategies 49
KV Cache Management 50
Prefix Caching 51
Tiered KV Cache (Offloading Beyond Local GPU Memory) 52
Cache-aware Request Routing 55
Prefill-Decode Disaggregation 56
Phase Characteristics 56
When to Use Disaggregation 57
Performance Impact 59
KV Cache Transfer: NIXL 59
Right-sizing and auto-scaling an inference system 61
1. Define the Workload 61
Model characteristics 61
Request characteristics 61
Concurrency requirements 62
Latency objectives 62
Availability and scaling requirements 63
Example workload definition 63
2. Memory Sizing: Can the Model Fit? 64
Model weights 64
KV cache 65
Model Sharding 66
Accelerator eligibility 66
3. Latency and Throughput sizing for given service requirements 67
Benchmarking representative workloads 68
From performance measurements to accelerator selection 69
4. Instance Selection and Sizing Framework 61
Compare cost-performance efficiency 69
Estimating accelerator count 70
Using public benchmarks for initial sizing 70
5. Auto Scaling and Capacity Management 71
Establishing baseline capacity 71
Horizontal scaling 72
Selecting scaling metrics 72
Request queue length 72

Latency-driven scaling 73
KV cache utilization 73
Scaling response time considerations 73
Reducing Scale-Out and Cold Start Latency 74
Scheduled scaling 74
Capacity availability considerations 75
Managed scaling options 75
Implementing security and responsible AI 76
Before moving to inference 76
Model assessment 76
Documentation preparation 77
Technical safeguards 77
Governance setup 77
After inference deployment 78
Continuous monitoring 79
Incident response mechanisms 79
Feedback collection mechanisms 79
AWS Partner Network 81
Resources 82
AWS service documentation 82
Other AWS resources 82
Other resources 82
Contributors 83
Contributors 83
Document history 84

Generative AI inference architecture and best practices on AWS

Oussama Kandakji, Mia Chang, Vivek Gangasani, Paco Gonzalez, Nicolas Jourdan, Christian Kamwangala, Marco Sgroi, and Hamza Usmani, Amazon Web Services

Organizations that deploy generative AI (gen AI) models such as large language models (LLMs) and multimodal generation models face critical challenges in serving production workloads at scale. These challenges span multiple dimensions including how to:

- Maintain consistent low-latency responses for real-time applications and user experiences.
- Implement dynamic capacity scaling to handle unpredictable traffic patterns.
- Optimize infrastructure costs.
- Ensure high availability across deployments.

This guide provides comprehensive prescriptive guidance on selecting and implementing appropriate AWS inference services, designing resilient architectures, and applying proven best practices. This guidance can help organizations achieve performant, cost-effective, and reliable gen AI deployments.

Intended audience

This guide is intended for the following:

- AI and machine learning (ML) teams who have trained gen AI models, like LLMs and multimodal generation models such as diffusion models, and want to deploy them for production inference
- Solution architects and technical leaders evaluating AWS inference options.
- Organizations transitioning from model development to production deployment.
- AWS Partners and system integrators providing inference solutions and implementation services.
- Technical decision-makers assessing infrastructure requirements for AI model inference.

To understand the concepts and recommendations in this guide, you should have a basic understanding of ML concepts and AWS services.

Objectives

The recommendations in this guide can help you achieve the following:

- Understand the differences between AI training and inference workloads and their unique challenges.
- Navigate the AWS inference stack (serverless, managed, and self-managed) and select the appropriate service based on key decision criteria.
- Learn about model and system optimization techniques for efficient inference deployment.
- Review recommended security controls and learn about responsible AI (RAI) practices for production inference workloads.
- Leverage the AWS Partner Network to accelerate time-to-market and model distribution.

Attachments

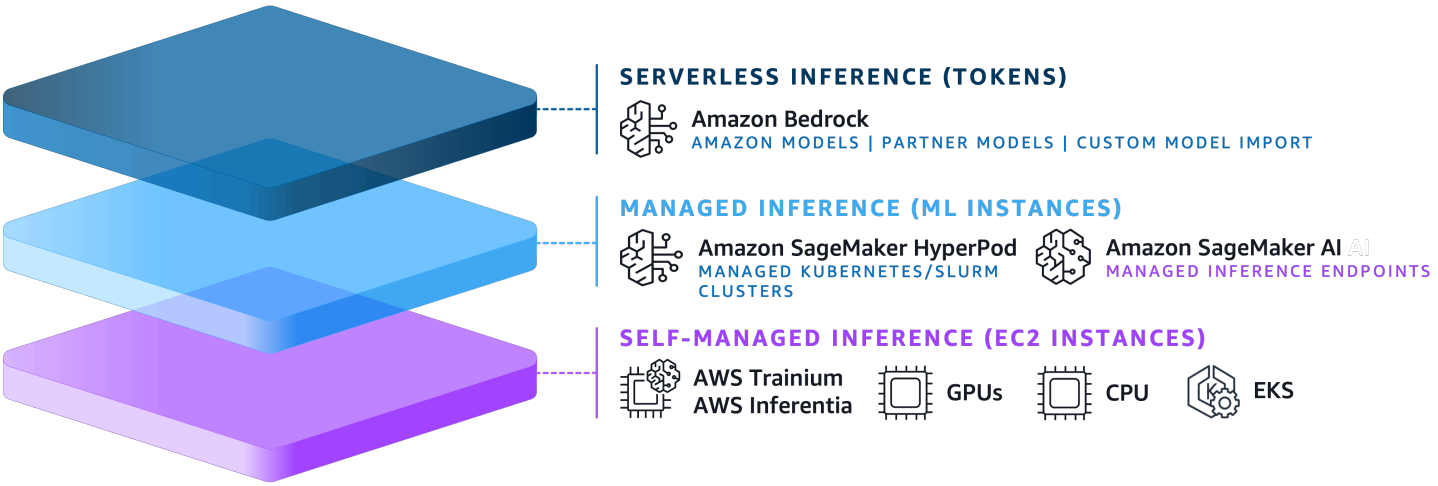
To access additional content that is associated with this document, download and unzip the following file:

attachment.zip

AWS inference stack

AI inference is the process of using a trained machine learning (ML) model to make predictions based on input data. Traditional ML inference typically involves relatively small models with modest compute requirements. However, generative AI (gen AI) inference has introduced new requirements in terms of compute, specialized hardware, and system optimizations.

As shown in the following diagram, AWS provides a multi-layer AI inference stack to address customers' diverse inference needs.



Following are descriptions of each layer of the AI inference stack.

Serverless inference layer

Following are key capabilities of the serverless inference layer, which is the top layer of the AI inference stack:

- Powered by Amazon Bedrock which offers a wide choice of industry leading foundation models (FMs) along with a broad set of capabilities to build generative AI applications.
- Supports importing custom models with popular architectures such as Llama, Mistral, and Qwen.
- Abstracts infrastructure management completely including high availability, scaling, and performance tuning.
- Pay-as-you-go pricing by using tokens or capacity units. For more information, see Amazon Bedrock pricing.
- Ideal for organizations who want to minimize operational and development effort.

Managed inference layer

Following are key capabilities of the managed inference layer, which is the middle layer of the AI inference stack:

- Powered by Amazon SageMaker AI which provides a broad set of tools to enable high-performance, low-cost ML for building, training and deploying AI models at scale. It provides all these capabilities in one integrated development environment (IDE).
- Enables serving pre-trained or custom models on managed infrastructure.
- Balances control with simplicity and with flexibility in choosing inference engines, scaling policies, and instance types.
- Suitable for organizations who want more control over deployment configuration, scaling behavior, and cost optimization while maintaining operational simplicity.

Self-managed inference layer

Following are key capabilities of the self-managed inference layer, which is the bottom layer of the AI inference stack:

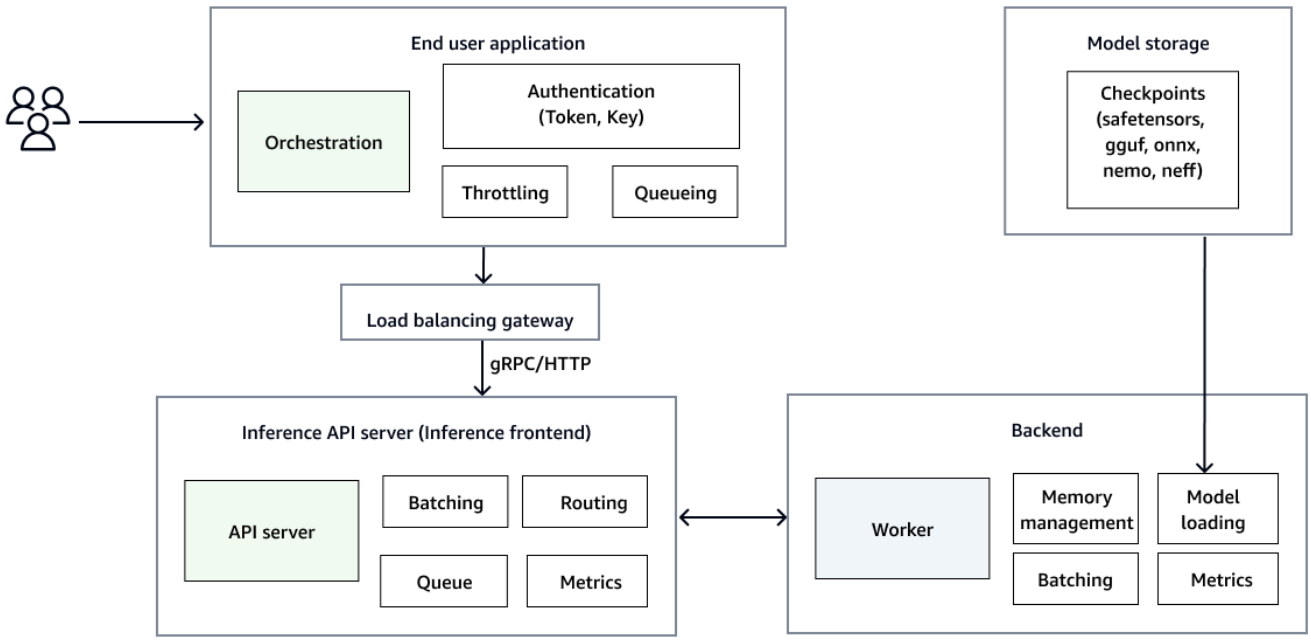
- Powered by Amazon EC2 instances with AWS Trainium and AWS Inferentia chips, AWS and NVIDIA GPUs, and CPUs. Offers broad and deep compute capabilities with over 1,000 instances and choice of the latest processor, storage, networking, operating system, and purchase model.
- Can operate through services like Amazon Elastic Kubernetes Service (Amazon EKS) or Amazon Elastic Container Service (Amazon ECS).
- Offers maximum flexibility and control of their infrastructure and software.
- Ideal for organizations who require complete control over their infrastructure.

Components of an AI inference stack

A production AI inference stack typically consists of the following three core components:

- End user application – Serves as the entry point for user interactions and request handling
- Inference API server (inference frontend) – Manages API serving and request processing
- Inference backend – Handles the core model execution and resource management

These components communicate through standardized protocols like gRPC for reliable and efficient model serving. The following diagram shows the high-level architecture of a real-time inference workload. It showcases the core components and how they communicate with each other.



This section provides details about each component.

End user application

The end user application component serves as the initial touchpoint for all client interactions, providing a secure and controlled entry point to the inference system. It incorporates essential security features like authentication through tokens and keys, implements throttling mechanisms to prevent system overload, and includes queuing capabilities to manage high request volumes. Some emerging tools use proxies and load balancers to enable key provisioning, multi-tenant or

multi-user throttling, queueing, and automatic failover in multi-region scenarios. Examples of these emerging tools include LiteLLM, Portkey, and Kong AI Gateway.

Inference API server or frontend

Acting as the bridge between client requests and model execution, the frontend component provides a robust API server that orchestrates the flow of inference requests. This component implements sophisticated request handling mechanisms including the following:

- Request batching for improved throughput
- Intelligent routing to distribute load across available workers
- Queue management for optimal request processing
- Comprehensive metrics collection to monitor system performance

The frontend communicates with the backend through standardized gRPC protocols, ensuring reliable and efficient data transfer. Popular frontends include NVIDIA Triton, NVIDIA Dynamo, Deep Java Library (DJL), Ray Serve, and Text Generation Inference (TGI) from Hugging Face.

Inference backend

The inference backend is where the model execution takes place, managed by specialized workers that handle the computational heavy lifting. The backend component is responsible for critical functions such as the following:

- Memory management to optimize resource utilization
- Model loading and initialization to ensure efficient startup
- Computation-level batching for improved throughput
- Performance metrics tracking

It works in close coordination with the frontend component to process requests while managing system resources effectively to maintain optimal performance and reliability. You can use a Python backend where you write your own inference logic. You can also use more optimized and popular backends such as vLLM, TensorRT LLM, ONNX Runtime, and SGLang. The backend component also maintains a connection to model storage, which houses various model formats including Safetensors, GGUF, ONNX, NeMo, NEFF, and Hugging Face models. This capability helps provide efficient access to the required model checkpoints.

Challenges of inference compared to training

Training large foundation models is often seen as the most resource-intensive phase of the AI lifecycle. However, moving to inference brings a different set of technical and operational challenges.

Training workloads are typically predictable, compute-bound, and throughput-oriented, whereas inference workloads are often more unpredictable, memory-bound, and latency sensitive.

This shift impacts every aspect of system design—from scaling and resource management to operational security. The following table outlines the key differences between training and inference challenges.

Category	Training	Inference
Operational complexity	• Planned, fixed-duration runs with predictable resource needs. • Scaling needs are clear in advance.	• Continuous or bursty workloads for real-time inference. • Autoscaling and traffic management and routing are essential to handle dynamic demand.
Latency and throughput	• Optimized for throughput. • Latency is secondary	• Latency-sensitive and concurrency-heavy. • Must deliver low time-to-first-token while supporting many simultaneous sessions.
Resource management	• Primarily compute-bound. • Large models stay loaded throughout the training. • Checkpoint writing must be handled efficiently.	• Often memory-bound because of model size and per-request state. • Requires memory-efficient serving techniques to prevent compute underutilization.

	• Node failure managemen t for long-running training workloads.	• Checkpoint loading must be efficient to enable fast scaling with dynamic demand.
Optimization strategies	• Focus on maximizing accelerator utilization and fast convergence • Optimizing data pipelines, using mixed precision. • Adaptive hyperparameters.	• Focus on minimizing per-request latency. • High efficiency and cost-per-request. • Leverage hardware accelerator architecture-aware optimizations, model compression, and caching to serve responses faster.
Security and compliance	• Controlled, internal environments with limited external exposure.	• Public or customer-facing endpoints. • Often requires tenant isolation and API key management. • Traffic encryption and compliance enforcement.
Cost control	• Predictable, capped costs tied to the length of the training run and the provisioned hardware resources	• Variable, usage-driven costs. • Risk of over-provisioning or idle spend because of request spikes. • Scaling must be carefully tuned to match demand patterns.

How AI Inference works

AI inference is the process of using a trained model to generate predictions or outputs from new input data. Inputs can be text, images, audio, video, or structured data — and outputs span generated text, images, audio, embeddings, classifications, or any combination of these. A single inference request can involve **multiple stages or even multiple models** working together to produce the final output. For example, a voice assistant may chain an automatic speech recognition (ASR) model, a large language model, and a text-to-speech (TTS) model in sequence. A retrieval-augmented generation (RAG) pipeline combines an embedding model, a re-ranker, and an LLM. An image generation workflow may pass through a text encoder, a diffusion model, and a super-resolution model.

Understanding how inference works at the model and the system level — what computations occur, where bottlenecks emerge, and what resources are consumed — is essential for making informed decisions about hardware selection, deployment architecture, and optimization strategy.

The transformer architecture

The transformer architecture, introduced in 2017, is the foundation behind the vast majority of modern AI models used in production inference today. Understanding how transformers work — and how different model families use them — is essential for reasoning about inference performance and optimization.

Core components

At its heart, a transformer is built from a stack of identical **layers**, each containing several components working together:

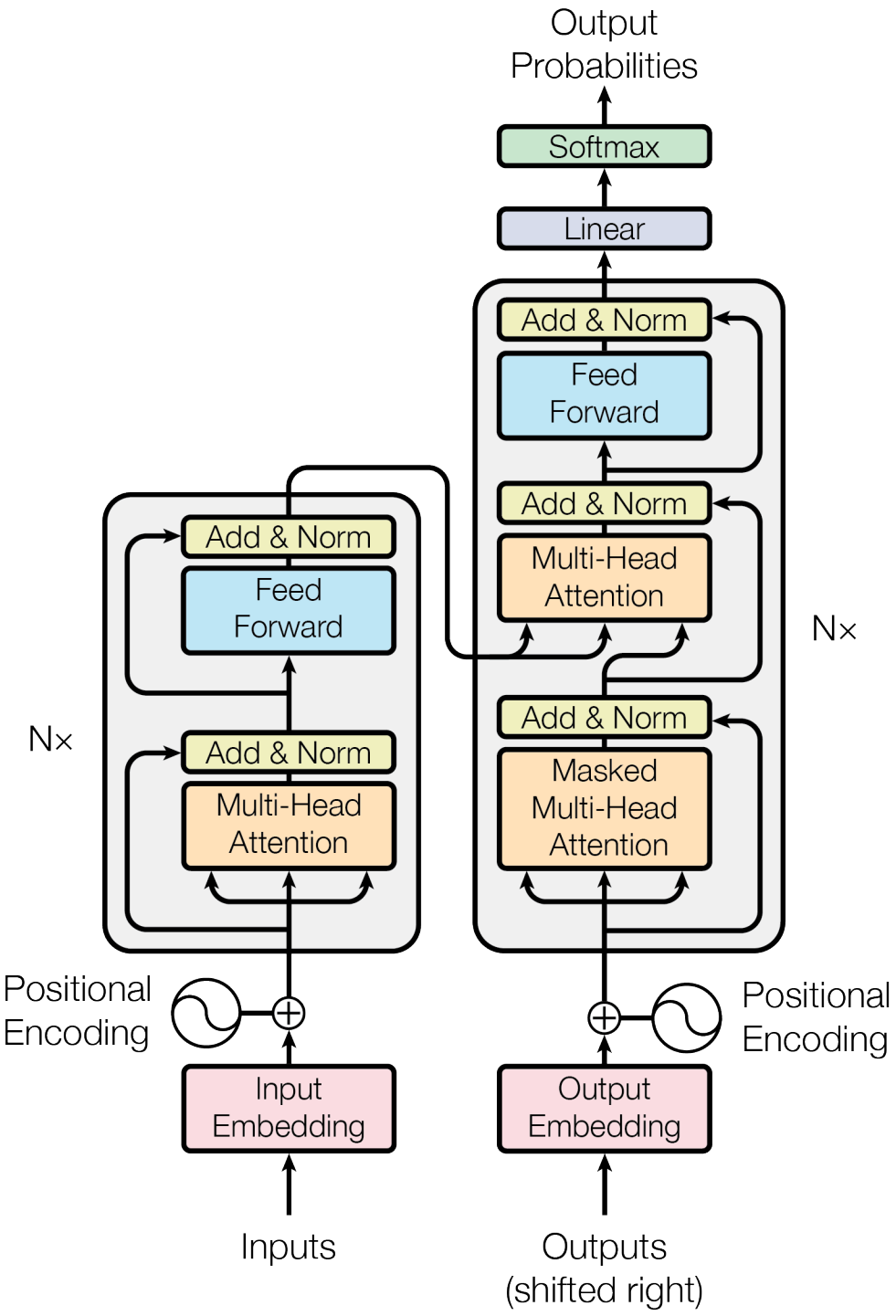
1. **Attention mechanism**: Allows the model to weigh the importance of different parts of the input when processing each element. For example, when processing the word "it" in a sentence, attention determines which previous words "it" refers to.
2. **Feed-forward network**: A neural network that processes each position independently after attention has mixed information across positions.
3. **Residual connections and normalization** ("Add & Norm"): After each attention and feed-forward step, the original input to that step is added back (residual connection), and the result is normalized. This helps training stability and allows information to flow through many layers.

Additionally, transformers use **embeddings** to convert input tokens (words, sub-words, or pixels) into numerical vectors, and **positional encoding** to inject information about the order of elements in the sequence (since attention itself has no inherent sense of position).

Each layer transforms its input (a set of vectors representing the data) and passes it to the next layer. The final layer's output is used to produce predictions — whether that's the next word, an embedding vector, or a classification score.

Encoder, decoder, and hybrid architectures

Transformers can be organized in different ways depending on the task. The diagram below shows the original transformer architecture from the 2017 "Attention Is All You Need" paper, which combines an **encoder** (left) and **decoder** (right):



The key architectural patterns are:

- **Encoder-only**: Uses just the left side. Processes the entire input at once using **multi-head attention** (bidirectional — each token can attend to all others). Produces a fixed output like an embedding vector or classification score. Examples: BERT, embedding models.

- **Decoder-only**: Uses just the right side. Processes input and generates output one element at a time using **masked multi-head attention** (causal — each position can only attend to previous positions, not future ones). This enables autoregressive generation. Examples: GPT, Llama, Qwen.
- **Encoder-decoder**: Combines both sides. The encoder processes the input bidirectionally, then the decoder generates output autoregressively while using **cross-attention** to access the encoder's output at each step. Examples: T5, BART, the original Transformer used for translation.

Model families and inference patterns

Different model families use these architectural patterns in different ways, resulting in fundamentally different inference characteristics:

Model Family	Architecture	Inference Pattern	Examples
Embedding models	Encoder-only	Single forward pass (vector output)	BERT, E5, GTE
Re-ranker models	Encoder-only or cross-encoder	Single forward pass (scoring)	BGE Reranker
Large Language Models (LLMs)	Decoder-only	Context processing + sequential generation	GPT, Llama, Mistral
Encoder-decoder models	Full transformer	Encode input + generate output	T5, BART, Whisper
Vision-Language Models (VLMs)	Vision encoder + decoder LLM	Encode image + process context + generate	Qwen-VL, Gemma3
Diffusion models	Transformer-based (DiT) or U-Net	Iterative refinement (denoising)	Stable Diffusion 3, Qwen-Image

Understanding this taxonomy matters because the inference characteristics — and therefore the optimization strategies — differ fundamentally based on which pattern a model follows. **The rest of this guide focuses on Large Language Models (decoder-only transformers)**, which are the most common and resource-intensive models deployed for inference today.

LLM inference: Prefill and decode

As established above, Large Language Models mostly use a decoder-only transformer architecture. This architectural choice means LLMs operate in two fundamentally different phases: **prefill** (processing the input context) and **decode** (generating output tokens one at a time). These two phases have completely different performance characteristics.

The prefill phase (context encoding)

When a request arrives, the LLM first processes the entire input prompt in a single forward pass. This phase takes the input tensor (tokenized prompt) and the model weights, moves them from **GPU high-bandwidth memory (HBM) into the tensor cores**, and processes all input tokens in parallel through every transformer layer.

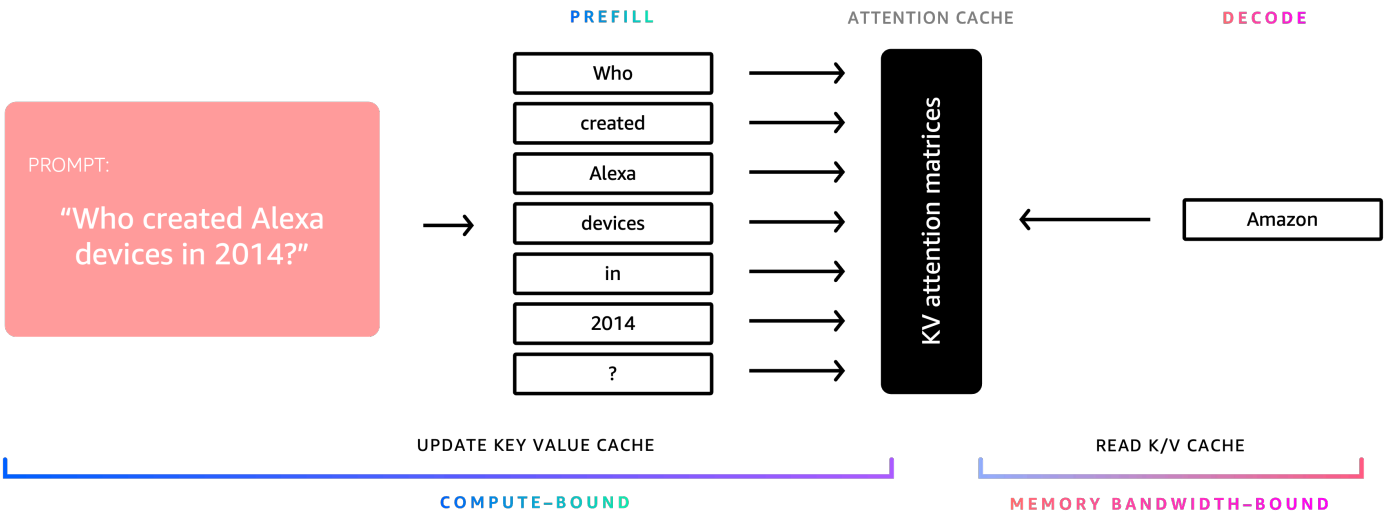
This phase computes and caches attention states for all input tokens, enabling efficient generation in the next phase.

Property	Detail
Data movement	Model weights + input tensor loaded from memory to compute cores**once**
Parallelism	High — all input tokens processed simultane ously
Performance constraint	**Compute-bound**(massive parallel matrix multiplications)
Output	Cached attention states for all input tokens + first output token
Metric impacted	**Time to First Token (TTFT)**

The decode phase (token generation)

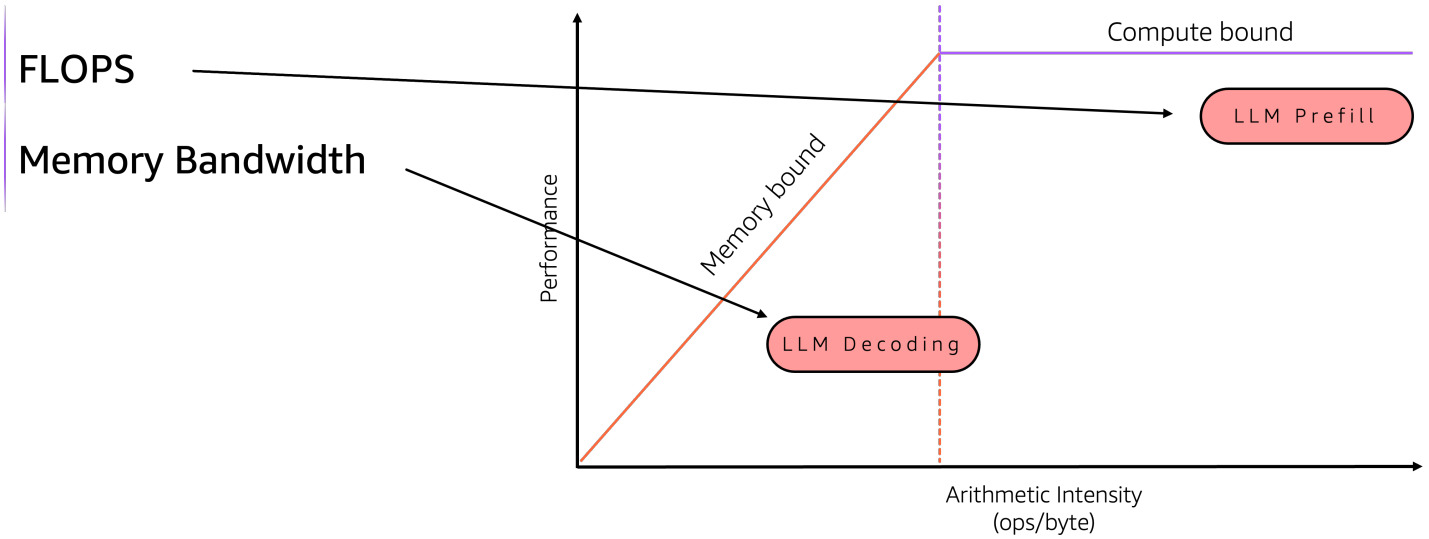
After prefill, the model switches to **autoregressive decode**: generating output tokens one at a time. Each decode step involves reading the model weights and the accumulated attention state from HBM back into the tensor cores — to produce just a single token.

Property	Detail
Data movement	Model weights + entire attention state moved from memory to compute cores **per token**
Parallelism	Low — each token depends on the previous
Performance constraint	**Memory-bandwidth-bound**(massive data movement for minimal compute)
Output	One token per step, with attention state updated
Metric impacted	**Time Per Output Token (TPOT),Tokens Per Second (TPS)**



The distinction between prefill and decode is fundamentally about **arithmetic intensity** — the ratio of compute operations (FLOPs) to memory accesses (bytes moved):

- **Prefill** has **high arithmetic intensity**: weights and inputs are loaded once, then many operations are performed across all tokens. The compute (FLOPS) is the bottleneck.
- **Decode** has **very low arithmetic intensity**: the entire KV cache and weights are loaded from memory for each step, but only enough compute to produce a single token is performed. The **memory bandwidth** (GB/s) is the bottleneck.



Optional: Vision encoding phase

Vision-Language Models (VLMs) extend the LLM inference pipeline with an additional encoding stage before prefill. They accept images (or video frames) alongside text and produce text outputs.

How VLM inference works

1. **Vision encoding**: A vision encoder (e.g., ViT, SigLIP) processes the input image into a sequence of visual tokens (patch embeddings)
2. **Projection**: Visual tokens are projected into the LLM's embedding space
3. **Prefill**: The combined sequence (visual tokens + text tokens) passes through the LLM backbone — this is standard prefill
4. **Decode**: Autoregressive token generation proceeds as normal

The KV cache

The **KV (Key-Value) cache** is the attention state stored in GPU memory that LLMs need to generate future tokens. It is one of the most critical concepts for understanding inference performance, memory consumption, and cost.

Why the KV cache exists

LLMs are autoregressive — each new token must attend to every previous token in the sequence. Without caching, the model would need to recompute the Key and Value projections for the entire context at every single decode step.

Since the Key and Value for a past token can be reused once computed, we can store them after computing them once. The KV cache stores these matrices so they are computed only once per token, dramatically reducing the computation required during decode.

Why only K and V? The Query represents "what the current token is looking for" — it's different for each new token and can't be precomputed. But the Keys and Values represent past tokens — they're fixed properties of those tokens, so we cache them.

How it works

1. During **prefill**, Key (K) and Value (V) projections are computed for every input token and stored in GPU HBM as the KV cache.
2. During **decode**, each new token:
   - Computes its own Query (Q), Key (K), and Value (V)
   - **Appends** its K and V to the cache
   - **Reads the entire cached K and V** to compute attention scores against all previous tokens
   - Uses the attention scores to create a weighted sum of cached Values
3. The cache grows linearly with sequence length — each new token adds one more K and V entry.

In the next section, we will cover the services on AWS that enable organizations to run AI inference at scale.

Selecting an AWS service for AI inference

When selecting an AWS service for AI inference, organizations need to consider multiple factors that align with their operational requirements and technical capabilities.

Key decision criteria to consider when choosing an AWS service include the following:

- Level of infrastructure management desired
- Existing compute commitments
- Pricing model preferences
- Required model architecture support
- Level of control needed in inference engine selection, configuration, and infrastructure

Relevant AWS services include Amazon Bedrock, Amazon SageMaker AI managed endpoints, Amazon SageMaker HyperPod, or self-managed infrastructure using Amazon EC2, Amazon EKS, and Amazon ECS.

Amazon Bedrock

Amazon Bedrock

Amazon Bedrock is a fully managed service that offers a choice of industry leading foundation models (FMs) along with a broad set of capabilities that you need to build generative AI applications and agents. Amazon Bedrock provides foundation model access, model customization, agent development, safety controls, and cost optimization — all behind a unified API. Since Amazon Bedrock is serverless, you don't have to manage any infrastructure, and you can securely integrate and deploy generative AI capabilities directly into your applications.

Infrastructure management

AWS manages the complete infrastructure lifecycle, including provisioning, scaling, and maintenance. This capability allows teams to focus on application development.

Pricing model

Amazon Bedrock offers flexible pricing based on API consumption. Organizations pay for input and output tokens processed during on-demand usage, or they can use and scale imported FMs measured in capacity units.

Bedrock exposes four inference service tiers — **Reserved**, **Priority**, **Standard**, and **Flex** — that let a workload trade off availability, latency, and cost without changing models or APIs. Three of them (Priority, Standard, Flex) are on-demand and share a single per-model quota; tier selection is just a request-time parameter. The fourth (Reserved) is a separate pool with its own SLA and minimum commitment. This sits alongside Bedrock's other capacity controls — Provisioned Throughput for custom and imported models, batch inference, and cross-region inference — and is usually the first lever to reach for on shared on-demand models.

Model architecture support

Amazon Bedrock provides access to hundreds of foundation models from leading AI companies. The catalog includes models from providers including OpenAI, Anthropic, Amazon, Meta, Mistral, and others. See Bedrock Models.

Beyond pre-built models, Bedrock supports:

- **Agent development**: Amazon Bedrock AgentCore is the end-to-end platform for building, connecting, and optimizing agents. Framework-agnostic and model-agnostic; no infrastructure management. Bedrock Managed Agents (powered by OpenAI for frontier models) are also available.
- **Customization**: Knowledge Bases for RAG, Bedrock Data Automation for unstructured content, prompt engineering, fine-tuning.

Automatic scaling

AWS automatically adjusts capacity based on demand patterns, which helps to provide consistent performance during traffic fluctuations.

Inference engine choice

Amazon Bedrock provides a fully managed inference engine optimized for the supported model architectures, with AWS handling all engine configuration and optimization.

Inference optimizations and configurations

You can control model behavior through model-specific request parameters. This capability allows customization of generation characteristics such as temperature, top-p sampling, and maximum token length for each inference request.

Inference Features

These features are available across the inference APIs:

- **Structured output**: validated JSON results.
- **Model reasoning**: extended thinking (where the model supports it).
- **Inference parameters**: temperature, top-p, max tokens, and other model-specific knobs.
- **Service tier selection**: `service_tier` request parameter accepting reserved, priority, default, or flex. Selects the capacity pool that serves the request. Resolved tier is reported back on the response, in CloudTrail, and as the ResolvedServiceTier CloudWatch dimension.

Cost Optimization

- **Intelligent Prompt Routing**: automatically route to the most cost-effective model that meets quality requirements.
- **Prompt caching**: reduce cost by reusing cached prompt prefixes.
- **Real-time and batch processing**: choose the processing mode that fits the latency and cost requirements.
- **Advanced Prompt Optimization**: optimize prompts for any model on Amazon Bedrock, while comparing your original prompts to optimized prompts across up to 5 models simultaneously.

Guardrails

Bedrock Guardrails is the safeguard layer that provides six configurable filter types that operate on inputs, outputs, or both. Guardrails can block up to 88% of harmful content, and the Automated Reasoning checks can identify correct responses with up to 99% accuracy. Guardrails can be applied during a regular inference call or invoked independently via the ApplyGuardrail API to protect non-Bedrock model traffic.

Supported clients and protocols

Applications can access models in Amazon Bedrock through the InvokeModel, Converse API. Applications can also connect to Amazon Bedrock via model provider specific APIs Responses API, Chat Completions API or Messages API. Further integration using the AWS SDK is also possible. These approaches provide straightforward integration paths for various application architectures. You can also use third-party open source libraries like Strands Agents or Crew AI to interact with Amazon Bedrock. You can also integrate third-party applications like Claude Code and CoWork from Anthropic or Codex from OpenAI.

Bedrock Endpoints

Bedrock exposes inference through two endpoints:

- **bedrock-mantle**: OpenAI Responses API, Anthropic Messages API.
- **bedrock-runtime**: Invoke API, Converse API.

Additional features available across endpoints: structured output (validated JSON), model reasoning (extended thinking), latency-optimized inference, and standard inference parameters (temperature, top-p, max tokens).

Amazon Bedrock — Inference APIs

Bedrock exposes inference through two endpoints:

- **bedrock-mantle** — OpenAI Responses API, Anthropic Messages API.
- **bedrock-runtime** — Invoke API, Converse API.

Additional features available across endpoints: structured output (validated JSON), model reasoning (extended thinking), latency-optimized inference, and standard inference parameters (temperature, top-p, max tokens).

API Summary

API	Endpoint	Shape
Responses API	bedrock-mantle	OpenAI Responses
Chat Completions API	bedrock-runtime	OpenAI Chat Completions

Messages API	bedrock-mantle	Anthropic Messages
Invoke API	bedrock-runtime	Bedrock-native
Converse API	bedrock-runtime	Bedrock-native, model-agnostic

Client and Tooling Compatibility

Because Bedrock exposes the OpenAI Responses and Anthropic Messages wire formats on the Mantle endpoint — alongside the native Converse and Invoke APIs on bedrock-runtime — many existing developer tools can route their inference through Bedrock with a configuration change rather than a code change. The qualifier matters: the client has to let you set the base URL (or have explicit Bedrock support) and accept Bedrock-issued credentials.

The operational consequence is what matters for procurement and security review: regardless of which API surface a client speaks, requests authenticate via *IAM*, can be reached over PrivateLink/VPC endpoints, are logged in CloudTrail, and are billed through the AWS account. Existing AWS controls — KMS, SCPs, Cost Explorer, Config — apply unchanged.

What this is **not**: Bedrock is not a drop-in replacement for OpenAI's or Anthropic's first-party APIs in every case. Feature-level coverage varies by model and by API (extended thinking, *service tiers*, structured output, prompt caching all have model-specific support matrices), and provider-specific endpoints outside the Responses / Chat Completions / Messages set are not part of the Bedrock surface. Validate the specific feature set your client depends on before migrating production traffic.

Inference Features

These features are available across the inference APIs:

- **Structured output** — validated JSON results.
- **Model reasoning** — extended thinking (where the model supports it).
- **Latency-optimized inference** — for time-critical workloads.
- **Inference parameters** — temperature, top-p, max tokens, and other model-specific knobs.
- **Service tier selection** — `service_tier` request parameter accepting reserved, priority, default, or flex. Selects the capacity pool that serves the request. Resolved tier is reported back on the response, in CloudTrail, and as the ResolvedServiceTier CloudWatch dimension.

Amazon Bedrock — Cross-Region Inference (CRIS)

Overview

Cross-region inference (CRIS) profiles let Bedrock route requests across AWS Regions to improve throughput. Two profile types — Geographic (within US/EU/APAC boundaries) and Global (any commercial Region) — trade data residency for cost and capacity. There is no additional routing cost; pricing follows the source Region.

Profile Types

Feature	Geographic	Global
Data residency	Within geographic boundaries (US, EU, APAC)	Any supported AWS commercial Region worldwide
Throughput	Higher than single-region	Highest available
Cost	Standard pricing	~10% savings vs. single-region
Best suited for	Organizations with data-residency regulations	Organizations prioritizing cost and performance

Operational Details

- **No routing cost** — the request is priced based on the Region from which the inference profile is called.
- **Account-level Region enablement is not required for routed Regions** — cross-region inference can route to Regions that have not been manually enabled in your account.
- **Network path** — all data transmitted during cross-Region operations remains on the AWS network. Traffic never traverses the public internet.
- **Encryption in transit** — data is encrypted between AWS Regions.
- **CloudTrail logging** — all cross-Region inference requests are logged in CloudTrail in your source Region. The additionalEventData.inferenceRegion field shows where each request was actually processed.
- **Provisioned Throughput** — not currently supported through inference profiles.

Amazon Bedrock — Guardrails

Overview

Bedrock Guardrails is the technical-safeguard layer that AWS Prescriptive Guidance recommends as part of an RAI program. It provides six configurable filter types that operate on inputs, outputs, or both. Guardrails can block up to 88% of harmful content, and the Automated Reasoning checks can identify correct responses with up to 99% accuracy. Guardrails can be applied during a regular inference call or invoked independently via the ApplyGuardrail API to protect non-Bedrock model traffic.

Six Filter Types

1. Content Filters

Detect and filter harmful text or image content in inputs or responses.

- **Predefined categories**: Hate, Insults, Sexual, Violence, Misconduct, Prompt Attack.
- **Configurable filter strength** per category.

Two tiers:

- **Classic** — base coverage.
- **Standard** — extends detection to code elements (comments, variable and function names, string literals).

2. Denied Topics

Define topics that are off-limits for the application. Blocks them in user queries or model responses. The Standard tier extends detection to code elements.

3. Word Filters

Block specific custom words or phrases (exact match). Includes a ready-to-use profanity option and supports custom word lists (e.g., competitor names).

4. Sensitive Information Filters

Block or mask PII and sensitive data (SSN, date of birth, address, etc.). Detection is probabilistic. Custom regex patterns supported for organization-specific PII.

5. Contextual Grounding Checks

Detect hallucinations in model responses that are not grounded in the provided source or are irrelevant to the user's query. Particularly useful for Retrieval Augmented Generation (RAG) applications.

6. Automated Reasoning Checks

Validate the accuracy of FM responses against a set of logical rules. Can detect hallucinations, suggest corrections, and highlight unstated assumptions. Up to 99% accuracy for correct responses.

Customization

Customize the messages returned to users when a filter is violated.

Usage Patterns

Two ways to apply Guardrails:

- **Inline during inference** — specify guardrail ID and version on the inference API call.
- **Standalone** — call the ApplyGuardrail API independently, without invoking a foundation model. Useful for protecting non-Bedrock model traffic or running guardrails on cached content.

For RAG and conversational applications, you can selectively evaluate only certain sections of the input prompt using tags.

Where Guardrails Fit in RAI

Guardrails maps to the **technical safeguards** layer of the AWS Prescriptive Guidance RAI playbook (input validation, output filtering, PII sanitization). It complements but does not replace *model assessment, documentation, governance, and access controls*.

Amazon Bedrock — Service Tiers and Reservations

Overview

Bedrock exposes four inference service tiers — **Reserved**, **Priority**, **Standard**, and **Flex** — that let a workload trade off availability, latency, and cost without changing models or APIs. Three of them (Priority, Standard, Flex) are on-demand and share a single per-model quota; tier selection is just a request-time parameter. The fourth (Reserved) is a separate pool with its own SLA and minimum commitment. This sits alongside Bedrock's other capacity controls — *Provisioned Throughput* for

custom and imported models, *batch inference*, and *cross-region inference* — and is usually the first lever to reach for on shared on-demand models.

The Four Tiers

Tier	Reservation	Pricing model	Best for
Reserved	Pre-purchased	Fixed $/1K TPM, billed monthly, 1- or 3-month term	Mission-critical, steady, high-volume workloads that can't tolerate downtime
Priority	None	Premium over on-demand	Latency-sensitive customer-facing flows that don't justify 24x7 reservation
Standard	None	Standard on-demand	Default for everyday production traffic
Flex	None	Discounted on-demand	Tolerant of longer processing times — evaluations, summarization, agentic workflows

Reserved Tier

Pre-purchased capacity with a 99.5% uptime target. You separately size input and output tokens-per-minute (TPM) to match the workload, controlling cost without overprovisioning. Minimum reservations:

- **Input TPM**: 100,000
- **Output TPM**: 10,000

When demand exceeds the reservation, requests automatically **overflow to the Standard tier**, so applications keep running rather than getting throttled. Commitment terms are 1 month or 3

months, billed monthly at a fixed $/1K TPM. Access is gated — contact your AWS account team to enable it.

Priority Tier

Fastest response times at a per-token premium over Standard on-demand. No reservation required. Set `"service_tier": "priority"` on the request and that call jumps to the front of the on-demand queue, ahead of Standard and Flex traffic. Use it for the customer-facing path; pair with Flex for background jobs in the same application to balance the bill.

Standard Tier

The default. Consistent performance for content generation, analysis, and routine document workloads. If service_tier is omitted, requests resolve to Standard. You can also explicitly set `"service_tier": "default"`.

Flex Tier

Discounted on-demand pricing in exchange for tolerating longer processing times. Aligned with the docs' guidance, the right candidates are model evaluations, content summarization, and agentic workflows where extra seconds don't change the outcome. Set `"service_tier": "flex"` on the request.

How Tier Selection Works

Tier is a per-request parameter on the runtime API:

```
"service_tier" : "reserved | priority | default | flex"
```

Observability:

- The chosen and resolved tier appear on the API response and in CloudTrail events.
- CloudWatch Metrics expose ModelId, ServiceTier, and ResolvedServiceTier. ResolvedServiceTier reflects what actually served the request — useful when reservations overflow to Standard, or when the requested tier isn't supported by a given model.
- Tier support is per-model; check the Models at a glance page for each model's supported tiers.
- IAM policies can restrict which tiers a principal can request.

Amazon SageMaker AI inference endpoints

SageMaker AI Inference endpoints deliver a managed inference environment purpose-built for deploying foundation models (FMs) and other ML models at scale, while giving teams control over the underlying instances, inference engine, and optimization strategy.

Infrastructure management

SageMaker AI manages the underlying ML instances while giving organizations control over instance types and configurations. The service handles instance provisioning, health monitoring, and infrastructure maintenance, so teams can focus on model deployment using containers maintained by AWS or custom containers. With inference components, you can deploy multiple FMs to a single endpoint, allocate accelerators, CPU, and memory to each model individually, and independently add, update, or remove models that share the same underlying infrastructure. To accelerate scaling of large generative AI models, SageMaker AI offers Container Caching, which pre-cache container images to reduce response times during traffic spikes. To update models safely in production, deployment guardrails provide controlled rollout options—including blue/green, canary, linear, and rolling deployments—with automatic rollback triggered by Amazon CloudWatch alarms to protect availability during endpoint updates.

Pricing model

Customers are charged based on the instance types deployed, with costs covering both compute resources and storage. This instance-based pricing model enables accurate cost forecasting based on instance selection and deployment duration. To optimize the cost of generative AI workloads with intermittent traffic, inference components can scale down to zero instances when idle and scale back up when requests arrive.

Model architecture support

SageMaker AI provides access to hundreds of foundation models through Amazon SageMaker JumpStart, including open-weight models such as Meta Llama, Mistral, DeepSeek, and Qwen, as well as NVIDIA NIM microservices, which you can deploy with a few clicks. The service also accommodates any compatible model framework or architecture, providing flexibility to deploy custom or fine-tuned generative AI models with PyTorch, vLLM, SGLang, or custom frameworks for multimodal and vision-language models.

Automatic scaling

You can configure automatic scaling policies that adjust the number of instances or model copies based on multiple factors. For example, factors include time schedules, built-in Amazon CloudWatch metrics such as invocation rates, concurrent requests, and resource utilization, or framework-specific metrics like Time to First Token (TTFT) tailored to specific application requirements. With capacity-aware inference and automatic instance fallback, you can define a prioritized list of instance types (instance pools) so that SageMaker AI automatically provisions from the next available option when a preferred instance type has insufficient capacity, which improves resilience during endpoint creation, scale-out, and scale-in. Per-instance-type CloudWatch metrics provide visibility into latency, throughput, GPU utilization, and instance count by hardware type within a single endpoint.

Inference engine choice

Teams can choose AWS Deep Learning Containers images that are optimized for popular frameworks and maintained by AWS. Alternatively, teams can bring custom container images that include specialized inference engines or dependencies.

Inference optimizations and configurations

The service enables flexible configuration of both model parameters and system-level settings. Generative AI optimization techniques, including the following:

- Quantization to lower-precision data types, such as FP8 and AWQ, to reduce hardware requirements
- Speculative decoding, which accelerates text generation without sacrificing output quality
- Model sharding and tensor parallelism for distributing large models across multiple GPUs or Trainium NeuronCores
- Engine-specific strategies such as prefix caching

For many models, you can deploy pre-optimized JumpStart models directly without running a separate optimization job. To identify the best deployment configuration, optimized generative AI inference recommendations automatically benchmark combinations of instance types (up to three at a time), serving containers, parallelism strategies, and optimization techniques on real GPU infrastructure against a performance goal (cost, latency, or throughput). You provide your model—base, custom, or fine-tuned, in Hugging Face/SafeTensors format from Amazon S3 or the

SageMaker AI Model Registry—along with your expected token distributions and concurrency, and SageMaker AI returns deployment-ready configurations with validated metrics, such as time to first token (TTFT), inter-token latency (ITL), P50/P90/P99 request latency, throughput, and cost. You can also benchmark existing production endpoints to right-size over-provisioned infrastructure and reduce recurring inference spend.

Supported clients and protocols

Applications can invoke endpoints through the Amazon SageMaker Runtime API actions over HTTPS, integrate using the AWS SDK, or use the Amazon SageMaker Python SDK for streamlined Python-based interactions. SageMaker AI endpoints also expose an OpenAI-compatible Chat Completions API, so you can invoke models—including those hosted as inference components—from the OpenAI SDK or frameworks such as LangChain and Strands Agents by changing only the endpoint URL, using time-limited bearer tokens for authentication. For interactive generative AI applications, you can use response streaming with the `InvokeEndpointWithResponseStream` action to return tokens as they are generated, and bidirectional streaming over HTTP/2 to stream data continuously in both directions for real-time, full-duplex use cases such as voice agents, live transcription, and speech-to-text.

Amazon SageMaker HyperPod

Amazon SageMaker HyperPod

Amazon SageMaker HyperPod with Amazon EKS orchestration enables large-scale, distributed model inference with enterprise-grade orchestration capabilities. It enables organizations to deploy, scale, and optimize foundation models with production reliability across the full model lifecycle from training to inference on a single unified compute infrastructure.

Infrastructure management

SageMaker HyperPod manages GPU and Trainium ML instances with automatic node replacement and deep health checks, while Amazon EKS provides Kubernetes orchestration with workload scheduling and namespace isolation. The HyperPod Inference Operator—installed as an EKS Add-on or via Helm—automates model deployment, autoscaling, observability, routing, and lifecycle management through two Custom Resource Definitions (CRDs): `JumpStartModel` for one-click deployment of models from SageMaker JumpStart, and `InferenceEndpointConfig` for full-control deployment of custom or fine-tuned models. Deployment interfaces include `kubectl apply` with CRD manifests, a Python SDK, Amazon SageMaker Studio UI, and the HyperPod

CLI. Networking uses Elastic Fabric Adapter (EFA) with GPU-Direct RDMA for high-bandwidth, low-latency inter-node communication and Application Load Balancers (ALB) for external traffic ingestion. Enterprise capabilities include custom Kubernetes pod configuration (init containers, volumes, scheduler), custom ACM certificates for endpoints (public, Private CA, or imported), Route 53 DNS management for custom domain names, per-pod request limits (max concurrent requests, queue size, overflow status codes), multi-instance type deployment with automatic failover across prioritized instance lists, custom node affinity for granular scheduling (AZ distribution, capacity type filtering, custom labels), EKS Pod Identity, and Multi-Instance GPU (MIG) support for deploying smaller models on fractional GPUs using NVIDIA MIG profiles (e.g., `mig-7g.40gb`, `mig-4g.71gb`). HyperPod also enables unifying training and inference infrastructure—the same cluster serves both workloads with Task Governance to govern compute resource usage across training, inference, and notebooks, and dynamic resource reallocation based on demand.

HyperPod provides comprehensive, one-click observability through AWS Distro for OpenTelemetry (ADOT) for metrics scraping, Amazon Managed Service for Prometheus for storage and querying, Amazon Managed Grafana with pre-built dashboards (inference metrics, tasks, cluster overview, and DPD-specific monitoring), and Amazon CloudWatch Container Insights for cluster-level visibility. Key metrics tracked include time-to-first-token (TTFT), inter-token latency (ITL), end-to-end request latency, GPU utilization and memory pressure, request queue depth, cache hit rates (L1/L2), and autoscaling events. For compliance, debugging, and model monitoring, inference data capture records inputs and outputs at three capture points—SageMaker AI Endpoint, Load Balancer (ALB access logs), and Model Pod—configurable via the `dataCapture` field in the CRD.

Pricing model

Customers are charged based on instance-based pricing covering compute and storage. SageMaker Training Plans offer commitment-based pricing with reduced rates and guaranteed capacity. With scale-to-zero, organizations pay nothing during idle periods when Karpenter removes all nodes. Unified infrastructure eliminates the need for duplicate clusters for training versus inference workloads.

Model architecture support

SageMaker HyperPod provides access to foundation models through Amazon SageMaker JumpStart, including open-weight and gated models, which can be deployed with a single `JumpStartModel` CRD. For custom or fine-tuned models, the `InferenceEndpointConfig` CRD supports loading from Amazon S3 buckets, Amazon FSx for Lustre, Hugging Face Hub, or local NVMe storage for reduced cold-start latency.

Automatic scaling

HyperPod implements a dual-layer autoscaling architecture. At the pod layer, KEDA (Kubernetes Event-Driven Autoscaling) is automatically installed with the Inference Operator and scales inference pods based on request queue length, CloudWatch metrics, latency, Prometheus metrics, or custom metrics—including support for scale-to-zero that eliminates compute costs during idle periods. At the node layer, Karpenter runs in the EKS control plane and dynamically provisions or removes compute nodes based on pending pod requirements, with intelligent instance type selection based on pod resource requests. Together, ADOT Collector scrapes metrics from inference pods and pushes them to Amazon Managed Prometheus or CloudWatch; KEDA evaluates metrics against thresholds and triggers HPA to create or remove pods; if new pods are pending due to insufficient capacity, Karpenter provisions nodes with appropriate GPU configurations; and when demand decreases, KEDA scales down pods while Karpenter consolidates workloads and removes underutilized nodes. The result is that organizations only pay for compute resources when actively serving inference requests.

Inference engine choice

Teams can choose AWS Deep Learning Containers optimized for generative AI serving engines, including vLLM, TGI, and SGLang. Alternatively, teams can bring custom container images with any inference framework or specialized dependencies.

Inference optimizations and configurations

The service enables advanced inference optimizations through managed tiered KV caching, intelligent request routing, and disaggregated prefill and decode:

- **Managed Tiered KV Cache** — A two-tier caching architecture stores intermediate attention computations from previous tokens. L1 cache uses CPU memory for low-latency local reuse on the same node. L2 cache offers two backend options: Redis for scalable node-level cache sharing across instances, or inbuilt SageMaker Managed Tiered Storage for higher performance in high-throughput LLM workloads. Performance gains include up to 40% reduction in latency, 25% improvement in throughput, and 25% cost savings versus baseline without these optimizations.
- **Intelligent routing strategies** — Routes requests based on configurable strategies: `prefixaware` routes requests with the same prompt prefix to the same instance (ideal for system prompts and shared contexts); `kvaware` routes to the instance with the highest KV cache hit rate (for mixed workloads with prefix overlap); `session` routes the same user session to

the same instance (for multi-turn conversations); and `roundrobin` provides even distribution without state (for stateless or uniform traffic).

- **Disaggregated Prefill and Decode (DPD)** — Separates the compute-bound prefill phase (processing the entire input prompt in parallel to generate the initial KV cache) from the memory-bandwidth-bound decode phase (generating tokens one at a time) onto dedicated GPU pools, removing interference that causes per-token latency spikes when colocated. The intelligent router tokenizes prompts and applies a configurable routing threshold to direct long prompts to prefiller pods and short prompts directly to decoder pods. KV cache transfer uses LMCache PD → NIXL → libfabric → EFA (GPU-Direct RDMA), with transfer costs of approximately single-digit milliseconds for 8,000 tokens on 3,200 Gbps EFA. Configuration is specified via a `pdSpec` field in the `InferenceEndpointConfig` CRD, with configurable routing threshold, independent prefill/decode replica scaling, per-role vLLM args, and intelligent routing strategy for multiple prefillers. Performance benchmarks with Llama 3.3 70B on P5 instances show 22–66% improvement in per-token latency (TPOT) on H100, 28–48% on H200; up to 35% improvement in output throughput on H100, up to 64% on H200; and 14–32% improvement in end-to-end P50 latency on H100, 29–41% on H200. Prerequisites include EFA-capable instances with RDMA read/write in the same Availability Zone (P5, P5e, P5en, P6 families).
- **Multi-Instance GPU (MIG)** — Partitions large GPUs into isolated instances, deploying models on specific NVIDIA MIG profiles to improve GPU utilization for smaller models, compatible with all other Inference Operator features.

Supported clients and protocols

Deployed inference workloads can be accessed through Application Load Balancers (ALB) over HTTPS, the Amazon SageMaker Runtime API, the AWS SDK, the Amazon SageMaker Python SDK, any OpenAI-compatible client (vLLM exposes `/v1/chat/completions`), or custom Route 53 DNS endpoints with ACM certificates.

Self-managed inference

Self-managed inference on Amazon EC2, Amazon EKS, or Amazon ECS provides organizations with complete control over their deployment environment and configuration.

Infrastructure management

Organizations maintain full control over infrastructure configuration and management. On Amazon EC2, teams manage instances directly. On Amazon EKS, Kubernetes provides container

orchestration. On Amazon ECS, AWS manages the container orchestration layer while organizations control task and service definitions.

Training compute reuse

Customers can expand or reallocate their existing Amazon EC2 capacity—currently used for non-inference tasks like training or evaluation—to support AI inference workloads.

Pricing model

Customers are charged based on provisioned compute instances, storage, and networking resources. On-Demand Capacity Reservations and Capacity Blocks for ML provide commitment-based pricing options that deliver reduced rates and guaranteed capacity for planned workloads.

Model architecture support

The environment accommodates any model format or architecture, enabling deployment of models built with any framework, custom architectures, or proprietary implementations.

Automatic scaling

Scaling mechanisms are tailored to each AWS service's capabilities:

- **Amazon EKS** – Kubernetes Horizontal Pod Autoscaler (HPA) or KEDA adjust pod replicas based on resource utilization or custom metrics. Also, for Amazon EKS, Karpenter dynamically provisions and scales nodes to match workload demands efficiently.
- **Amazon ECS** – Service autoscaling adjusts task counts based on CloudWatch metrics, target tracking policies, or step scaling configurations.
- **Amazon EC2** – Autoscaling groups manage instance fleets using dynamic scaling policies, predictive scaling, or scheduled actions.

All approaches can respond to built-in metrics (such as CPU, memory, and network) and to engine-specific performance indicators (such as request latency, queue depth, and throughput). They can also respond to custom metrics tailored to specific application requirements from Amazon CloudWatch, Amazon Managed Service for Prometheus, or third-party sources.

Inference engine choice

You can select from Deep Learning Container images that are optimized for popular frameworks and maintained by AWS. Or, they can deploy custom container images with any inference engine or specialized software stack.

Inference configuration

The environment provides complete customization of system configuration. Organizations can implement any quantization level, configure model sharding strategies, enable prompt caching mechanisms, define batching behaviors, select communication protocols, and optimize for specific performance characteristics.

Supported clients and protocols

Inference API servers can be exposed through Application Load Balancers, Network Load Balancers, or Kubernetes ingress controllers or envoy proxies. All approaches support any protocol or client compatible with the deployed inference engine or container. This flexibility enables integration with diverse application architectures and client requirements.

Model optimization techniques

Model optimizations focus on making the model itself faster and more compute and memory efficient — independent of the serving infrastructure. These techniques modify how the model is stored, compiled, or executed at inference time.

Model Compression

Model compression techniques reduce memory footprint and computational requirements by removing redundant information or reducing numerical precision (quantization). These methods can be applied independently or in combination to achieve optimal efficiency.

Quantization (Precision Reduction)

Quantization reduces the numerical precision of model weights (and optionally activations) from higher-precision formats (FP32, FP16/BF16) to lower-precision representations (FP8, MXFP4, NVFP4). This directly reduces:

- **Memory footprint** — Smaller weights mean the model fits in less GPU HBM
- **Memory bandwidth consumption** — Less data to move from HBM → compute cores per inference step
- **Compute latency** — Lower-precision arithmetic is faster on modern accelerators (e.g., Tensor Core FP8/INT8 throughput is 2× FP16)

The trade-off is potential quality degradation — though modern quantization techniques minimize this through calibration and mixed-precision strategies.

> **Tip**
>
> Prefer pre-quantized checkpoints where available, so scale factors are computed offline instead of at runtime.

Precision hierarchy

Format	Bits per param	Memory (7B model)

FP32	32	~28 GB
FP16 / BF16	16	~14 GB
FP8 (E4M3/E5M2)	8	~7 GB
4-bit (GPTQ/AWQ/MXFP4/NVFP4)	4	~3.5 GB

In addition to the above, newer generations of accelerated compute architectures natively support lower precisions. For example, AWS Trainium3 natively supports MXFP4 computation, and NVIDIA Blackwell's fifth-generation Tensor Cores natively support NVFP4.

KV Cache Quantization

Not all compression needs to target model weights. **KV Cache optimization** specifically reduces the memory consumed by the attention cache during inference — which can dominate memory usage for long sequences and large batch sizes.

KV Cache Quantization

- **KV-only quantization**: Compresses the Key and Value matrices in the attention cache to FP8 or INT8 while keeping model weights at full or higher precision. This preserves model quality while dramatically reducing the memory consumed per active request
- **Weight-only quantization**: Quantizes model weights but keeps activations and KV cache at higher precision. Reduces model loading time and static memory footprint while maintaining decode quality

Impact on capacity: For a model serving many concurrent requests with long contexts, KV cache often consumes more memory than the model weights themselves. Quantizing KV cache from FP16 to FP8 reduces the amount of memory necessary for KV blocks by half.

Model compilation

Model compilation optimizes how operations are executed on hardware. At inference time, the choice between **eager mode** (operation-by-operation execution) and **compiled/graph mode** (pre-

optimized execution paths) significantly impacts latency and throughput. Inference engines like vLLM, SGLang, and TensorRT-LLM leverage these compilation modes to maximize performance.

Execution Modes: Eager vs. Graph

Eager Mode:

- Operations execute immediately as called, one at a time
- Flexible — supports dynamic shapes and easy debugging
- Higher overhead — each operation dispatched individually from CPU to GPU

Graph/Compiled Mode:

- Operations are captured into a reusable execution graph during warm-up
- Entire graph executes in one dispatch, eliminating per-operation overhead
- During graph capture, inference engines apply hardware-optimized kernel implementations — for example, vLLM selects specialized kernels for MoE expert computation (DeepGEMM, CUTLASS, FlashInfer), fused attention kernels (FlashAttention, PagedAttention), and quantized GEMM operations tailored to the target hardware and precision format

Example: vLLM uses CUDA graphs by default for maximum throughput, but provides the `--enforce-eager` flag for flexibility.

Kernel Optimization During Compilation

When inference engines compile a model into a CUDA graph or execution plan, they replace generic operations with specialized, fused kernels optimized for the target hardware and model architecture. This kernel selection happens at graph capture time and can significantly impact performance.

Key kernel categories applied during compilation:

- **MoE expert kernels**: For Mixture-of-Experts models, engines select from specialized implementations like DeepGEMM (FP8-optimized), CUTLASS (NVIDIA's high-performance GEMM library with FP4/FP8 variants), or FlashInfer (minimal memory overhead for FP4/FP8). Selection depends on quantization format, activation function, and parallelism strategy.

- **Attention kernels**: Fused implementations like FlashAttention-2 or PagedAttention that combine multiple attention operations into single kernel launches, reducing memory bandwidth and improving arithmetic intensity.
- **Quantized GEMM operations**: Hardware-accelerated kernels that leverage Tensor Cores for mixed-precision computation (FP8, FP4, INT8).

These optimizations leverage NVIDIA GPU capabilities like Tensor Cores for maximum throughput. While kernel selection typically happens automatically based on model configuration and hardware detection, inference frameworks expose parameters to manually enable or disable specific kernel implementations — useful for debugging performance issues or testing different backends for your workload.

Trainium Compilation with Neuron SDK

AWS Trainium leverages compilation using Neuron SDK in order to prepare and optimize the model for its execution on its purpose-built accelerator cores. Models can be compiled before deployment using different libraries like Optimum Neuron, NeuronX Inference or PyTorch, which all use the Neuron compiler and leverage Neuron Kernel Interface (NKI) to apply custom and optimized kernels. The ecosystem is evolving rapidly — consult the Neuron documentation for latest news and best practices.

Speculative decoding

Speculative decoding is an acceleration technique for autoregressive generation (token generation). Rather than producing one final token per expensive model step, it uses a cheaper proposal mechanism to suggest several candidate tokens, then verifies them with the target model in a single pass.

Core principle: decode-time generation is usually memory-bandwidth bound, so the target model can often score multiple proposed tokens with only a modest increase in cost compared with scoring one token. If proposals are cheap enough and acceptance is high, the system can return multiple tokens per target-model step, often delivering substantial speedups with no change in output quality.

All speculative decoding variants follow the same basic loop:

1. **Proposal phase**: generate candidate tokens using a fast mechanism.

2. **Verification phase**: run the target model to evaluate those candidates.
3. **Acceptance**: keep the longest left-to-right prefix that matches the target; stop at the first mismatch and continue from there.

The techniques differ in how they produce proposals, which can be grouped into three categories:

Model-less Speculation

Technique: Pattern matching against already-generated text — no separate model needed.

How it works: Builds a suffix tree or n-gram statistics from generated text and the prompt, then proposes continuations from matching patterns when similar contexts reappear. The target model verifies these pattern-based candidates.

Best for:

- Structured outputs (JSON, YAML, tool calls)
- Code generation with repetitive patterns
- Agentic workflows with repeated actions

Advantages:

- Zero memory overhead — no additional model
- Zero setup — works with any model out of the box
- Effective for constrained or repetitive generation

Example: Suffix-based speculative decoding uses a **suffix tree** built from the prompt tokens to speculatively predict future tokens (matching repeated patterns/suffixes in the input), configured in vLLM via `--speculative-config '{"method": "suffix",...`

Draft Model Speculation

Technique: Use a smaller, faster model to generate candidate tokens.

How it works: A small draft model generates K tokens autoregressively, then the target model validates all candidates in one pass. Tokens are accepted until the first disagreement, where the target model's prediction replaces the rejected token.

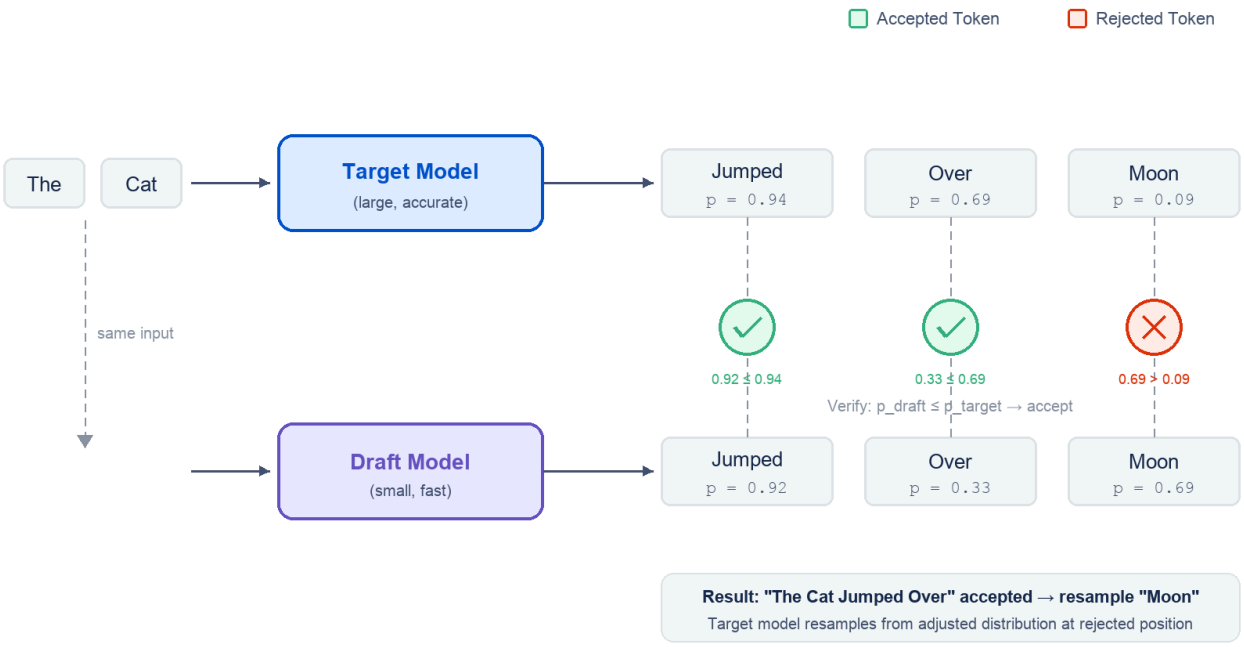
Best for: General-purpose generation when a compatible draft model is available (shared vocabulary and training distribution)

Advantages:

- Proven approach with broad applicability
- Works across diverse content types
- Draft model can be reused across workloads

Trade-offs:

- Requires loading a separate draft model (additional memory)
- Acceptance rate depends on draft-target alignment



Configuration example (vLLM):

```
vllm serve meta-llama/Llama-3.1-70B-Instruct \
  --speculative-config '{"model": "meta-llama/Llama-3.2-1B-Instruct",
 "num_speculative_tokens": 5}'
```

Decoding Head Speculation

Technique: Train and/or use a lightweight "draft head" that predicts tokens using the target model's internal hidden states.

How it works: A small neural network (1–2 transformer layers) is trained to predict next-token distributions from the target model's latent features. At inference, the EAGLE or Medusa head generates candidates using the target model's own representations from the previous step, achieving higher acceptance rates than independent draft models.

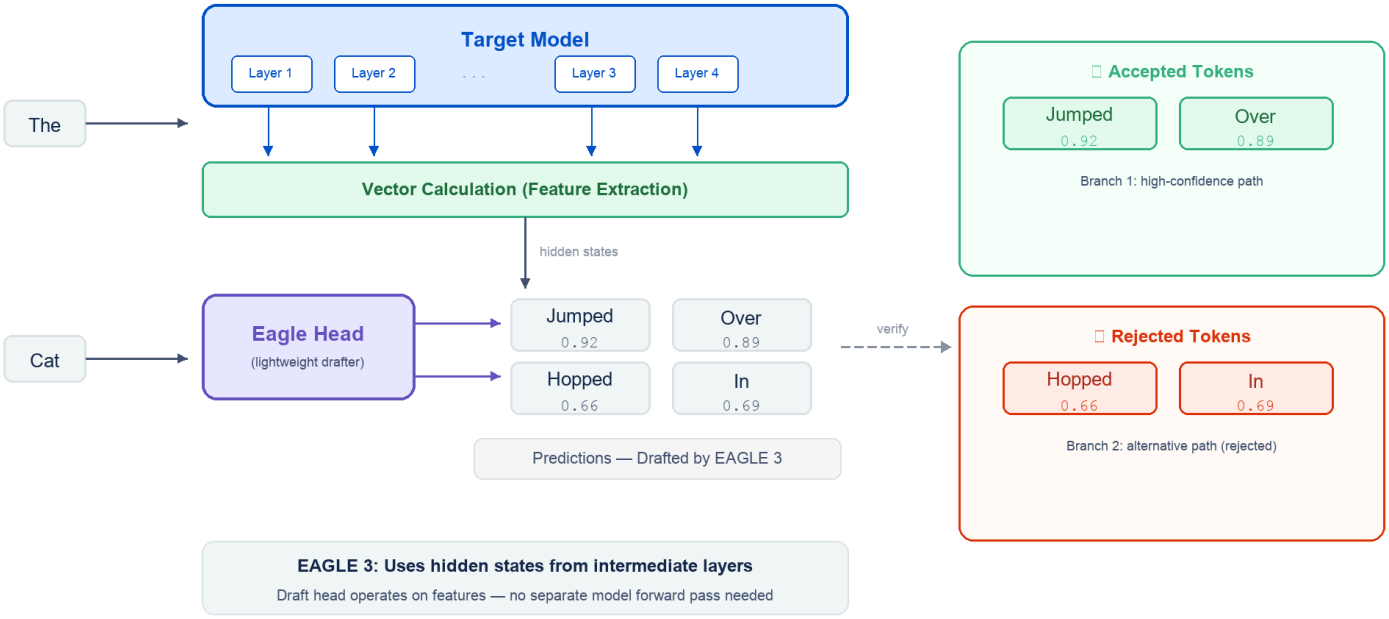
Best for: Maximum speedup when a trained head is available for your target model

Advantages:

- Higher acceptance rates than draft models (leverages target's internal representations)
- Minimal memory overhead (~1% of target model size)
- Better prediction alignment with target model behavior

Trade-offs:

- Requires training a speculator head for each target model if one is not already available
- More complex setup than other approaches



Configuration examples

In SageMaker AI, you can train EAGLE3 speculative decoder heads with SageMaker-curated datasets or with your own data. You can also leverage EAGLE3 speculative decoding using SageMaker AI optimized generative AI inference recommendations. In addition, the following configuration shows how to use EAGLE3 and NVFP4 quantization in vLLM.

```
vllm serve nvidia/Kimi-K2.6-NVFP4 \
  --trust-remote-code \
  --tensor-parallel-size 4 \
  --tool-call-parser kimi_k2 \
  --enable-auto-tool-choice \
  --reasoning-parser kimi_k2 \
  --attention-backend tokenspeed_mla \
  --speculative-config '{"model":"lightseekorg/kimi-k2.6-eagle3.1-
mla","method":"eagle3","num_speculative_tokens":3}' \
  --language-model-only
```

Multi-LoRA Serving

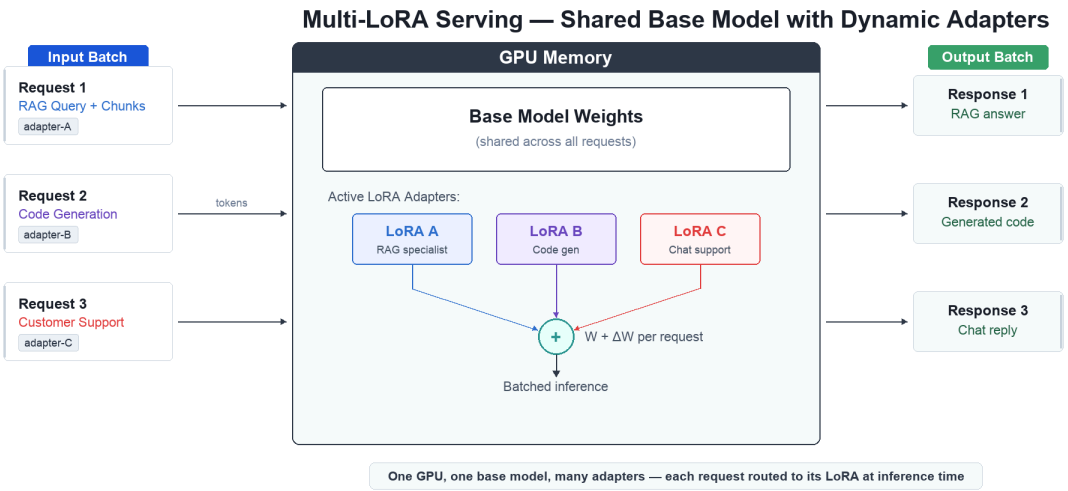
Multi-LoRA serving enables a single base model to serve multiple LoRA (Low-Rank Adaptation) adapters simultaneously, allowing different requests to use different fine-tuned variants without maintaining separate model deployments. LoRA adapters are small weight matrices (typically <1% of base model size) that modify specific layers to specialize model behavior for particular domains, tasks, or users.

How Multi-LoRA Serving Works

Rather than deploying N separate fine-tuned models, multi-LoRA serving loads one base model in GPU memory and keeps multiple lightweight LoRA adapters available. Incoming requests specify which adapter to use via API parameter, and the inference engine dynamically applies the appropriate adapter weights during forward pass.

Key characteristics:

- **Shared base model**: One copy of base weights (~14GB for a 7B FP16 model) serves all adapters
- **Lightweight adapters**: Each LoRA adapter is typically 10-100MB depending on rank and target modules
- **Dynamic switching**: Different concurrent requests can use different adapters in the same batch
- **Automatic management**: Engine handles adapter loading/unloading based on memory constraints and request patterns



Memory and Performance Impact

Multi-LoRA serving trades a small amount of compute overhead (applying adapter weights) for dramatic memory savings compared to deploying separate models:

- **Memory**: Base model (14GB) + N adapters (50MB each) vs. N full models (14GB each) — enabling 100+ adapters where only 7-8 full models would fit
- **Latency**: Minimal overhead per request (~5-10% compared to base model) when adapters are cached in GPU memory. Higher latency if adapter needs loading from CPU/disk
- **Resource Utilization**: Higher as it prevents replicating base-model weights for each separate task-specific model

When to Use Multi-LoRA Serving

Multi-LoRA serving is most valuable when you need to serve many variants of a base model:

- **Multi-tenant deployments**: Different customers or teams with domain-specific fine-tunes sharing infrastructure
- **Task specialization**: Single endpoint serving adapters for different capabilities (summarization, extraction, code generation)
- **Personalization**: User-specific or group-specific adapters for customized behavior
- **A/B testing**: Concurrent evaluation of multiple fine-tune candidates against production traffic
- **Domain adaptation**: Industry-specific variants (legal, medical, finance) without separate deployments

Configuration Examples

In SageMaker AI, you can leverage the efficient multi-adapter inference as a native feature. More details in this blog post. In addition, the following configuration shows how to use multi-adapter LLM inference natively in vLLM.

```
# Start server with multi-LoRA support
vllm serve Qwen/Qwen3.6-35B-A3B \
    --enable-lora \
    --enable-mixed-moe-lora-format \
    --tensor-parallel-size 4 \
```

```
    --enable-expert-parallel \
    --lora-modules \
        '{"name": "lora-2d", "path": "jeeejeee/qwen36-35ba3b-2d-weights-poken-lora",
 "is_3d_lora_weight": false}' \
        '{"name": "lora-3d", "path": "jeeejeee/qwen36-35ba3b-moe-all-linear-poken-
lora", "is_3d_lora_weight": true}'

# Request with specific LoRA adapter
curl -X POST http://localhost:8000/v1/load_lora_adapter \
-H "Content-Type: application/json" \
-d '{
    "lora_name": "lora-3d",
    "lora_path": "/path/to/3d-format-lora",
    "is_3d_lora_weight": true
}'
```

Applying inference optimizations in SageMaker AI

SageMaker AI leverages the techniques described earlier. With SageMaker AI, you can improve the performance of your generative AI models by applying inference optimization techniques. By optimizing your models, you can attain better cost performance for your use case. When you optimize a model, you choose which supported optimization technique to apply, including quantization, speculative decoding, compilation, and fast model loading. After your model is optimized, you can run an evaluation to see performance metrics for latency, throughput, and price.

System-level optimizations

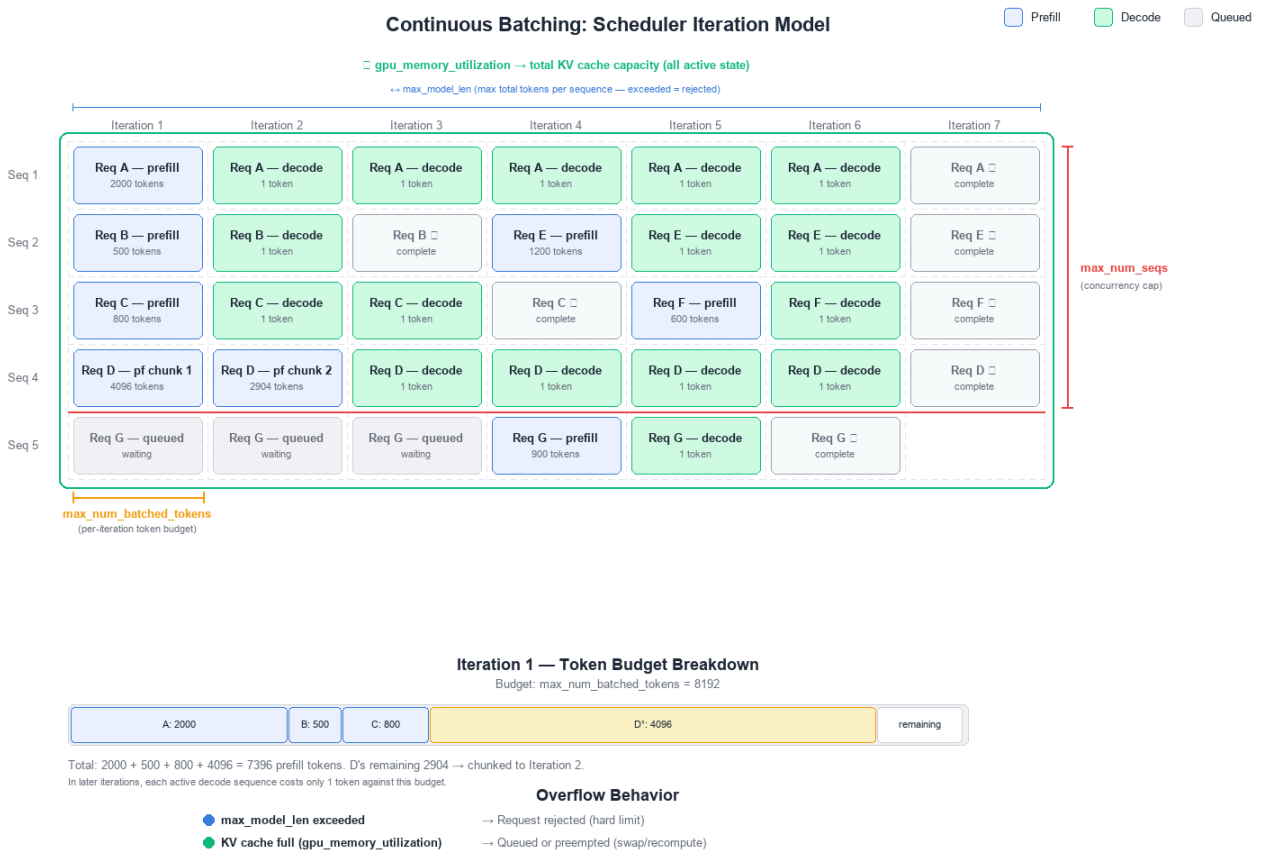
System optimizations focus on **how the inference serving infrastructure is configured and orchestrated** — independent of the model itself. These techniques operate at the serving engine, cluster, and request-scheduling layers to maximize hardware utilization, reduce latency, and increase throughput.

Inference Engine Tuning

Modern inference engines like vLLM, SGLang implement **continuous batching** (iteration-level scheduling) — dynamically admitting and removing requests at every decode step rather than processing fixed batches. As soon as one request completes, a new one fills its slot. This eliminates padding waste and idle GPU time, but shifts tuning from "pick a batch size" to "configure scheduler guardrails" that define capacity limits, memory allocation, and fairness policies.

Anatomy of Inference Requests

The diagram below shows how the scheduler works at the iteration level. Each **iteration** (column) is one forward pass processing both prefill tokens (from new requests) and decode tokens (from active requests). Sequences flow through prefill (blue) → decode (green) → complete, with overflow going to queue (gray).



Core Parameters and Their Implications

`max_model_len` **(sequence length limit)**: Sets maximum total tokens (prompt + output) per request. This determines per-request KV cache memory consumption. A 128K limit reserves ~256MB per request; an 8K limit reserves ~16MB — allowing 16× more concurrency from the same memory pool. Setting this too high **wastes memory** and **reduces throughput**. Too low causes **hard rejections** (HTTP 400) — the only parameter that fails requests rather than queuing them. **Context impact**: longer sequences also increase prefill/decode latency, though batching mitigates this. Tune to workload P95/P99, not theoretical max. *(vLLM:* `--max-model-len`*, SGLang:* `--context-length`*)*

`gpu_memory_utilization` **(memory allocation fraction)**: Fraction of GPU memory reserved at startup. After model weights and activations (fixed), the remainder becomes the **KV cache pool**. Higher values boost **throughput** by fitting more concurrent requests, but risk **OOM errors** if transient CUDA allocations spike. When KV memory is exhausted, the system **queues new requests** (increased TTFT) or **preempts active ones** by swapping KV cache to CPU or discarding for recomputation (10-100× latency spikes from thrashing). Start at 0.85-0.90; reduce by 0.05 if you

see preemption events or OOM crashes. *(vLLM:* `--gpu-memory-utilization`*, SGLang:* `--mem-fraction-static`*)*

`max_num_seqs` **(concurrency cap)**: Maximum requests active simultaneously (prefill + decode). Higher values improve **throughput** and GPU utilization but increase **per-request latency** (more contention for compute) and **queuing delays**. Lower values reduce **latency variance** but may **underutilize the GPU** during bursty traffic. Long-context requests consume many KV blocks, so effective concurrency may hit memory limits before this cap. When reached, new requests **queue** (FCFS). *(vLLM:* `--max-num-seqs`*, SGLang:* `--max-running-requests`*)*

`max_num_batched_tokens` **(per-iteration budget)**: Total tokens processed per forward pass. Consumed by prefill tokens (thousands per new request) and decode tokens (1 per active sequence). Effectively a **prefill admission rate limiter**. Higher budgets improve **prefill throughput** (lower TTFT, better GPU utilization), but long prefills can monopolize the GPU and cause **decode stalls** (latency spikes for other requests). Lower budgets prevent stalls but slow prefill. Without chunked prefill, prompts exceeding the budget **queue**. With chunking, they split across iterations. Balance: 4096-8192 tokens. *(vLLM: `--max-num-batched-tokens`)

Enabling Chunked Prefill

The problem: A 32K-token prefill processed in one iteration monopolizes the GPU for hundreds of milliseconds, severely limiting concurrent decode operations. Without chunking, decode requests can only advance by **a single token** during the entire prefill, causing decode streams to effectively pause. These **decode stalls** appear as P95/P99 TPOT spikes whenever long-context requests arrive.

The solution: Chunked prefill splits long prompts into smaller chunks (configurable, typically 512-8192 tokens) and **prioritizes decode requests between chunks**. Instead of one decode step during an entire prefill, there can now be as many decode steps as there are prefill chunks. In the diagram, Req D's prefill spans iterations 1-2 (4096 + 2904 tokens) rather than blocking iteration 1 entirely. The per-iteration token budget (`max_num_batched_tokens`) determines max chunk size: if 200 requests are decoding (200 tokens) and `max_num_batched_tokens=8192`, then 7992 tokens remain for prefill chunks that iteration.

Why it helps: Without chunking, long prefills cause complete decode pauses for concurrent requests. Chunking transforms this into **gradual slowdown instead of total blocking**, while maximizing GPU utilization by combining compute-bound (prefill) and memory-bound (decode) work in the same batch. Trade-off: **slight TTFT increase** (chunked prefill has some overhead vs. contiguous prefill) for **major ITL improvement** and **throughput gains** (benchmarks show +50%

token throughput improvement on concurrent workloads). The chunk size provides a tuning knob: smaller chunks prioritize decode fairness; larger chunks (>2048) prioritize throughput.

When to enable: Recommended as default for most production deployments with concurrent traffic. The benefits are highest when prefill and decode operations overlap. Disable only for single-user or offline batch scenarios where fairness doesn't matter. *(vLLM: `--enable-chunked-prefill --max-num-batched-tokens 4096`, SGLang: `--chunked-prefill-size`).

Parallelism

Parallelism strategies determine how a model is distributed across multiple GPUs. This is the foundation of your serving architecture — all other optimizations (batching, caching, disaggregation) operate on top of your parallelism topology.

Parallelism Strategies

Tensor Parallelism (TP): Splits each model layer across multiple GPUs. All GPUs work on the same request simultaneously.

- **Why**: Fits large models in memory, reduces per-request latency
- **Trade-off**: Requires high-bandwidth interconnects (NVLink/NVSwitch/EFA). More GPUs = more communication overhead.
- **When**: Model doesn't fit on a single GPU, or you need lower per-request latency
- **vLLM config**: `--tensor-parallel-size=4` (splits across 4 GPUs)

Expert Parallelism (EP): For Mixture-of-Experts (MoE) models, distributes experts across GPUs.

- **Why**: MoE models have too many experts to fit all on one GPU
- **Trade-off**: Only active experts for each token require communication
- **When**: Serving MoE models (Mixtral, DeepSeek, Qwen MoE)
- **vLLM config**: `--enable-expert-parallel` (by default, expert layers form a TP group of size DP × TP without this flag)

Data Parallelism (DP): Replicates the entire model (or TP/EP group) across multiple independent serving groups. Each DP replica processes its own batch of requests with its own KV cache.

- **Why**: Scale throughput linearly without affecting per-request latency
- **Trade-off**: Uses more total GPUs. For dense models, replicas are fully independent (no communication). For MoE models with EP enabled, replicas communicate via AllToAll for expert dispatch/combine.
- **When**: Single TP/EP group cannot handle your throughput requirements
- **vLLM config**: `--data-parallel-size=4` (creates 4 independent replicas)

Combining parallelism strategies: DP can be layered with TP or EP. Example: `--tensor-parallel-size=2 --data-parallel-size=4` requires 8 GPUs total (4 replicas × 2 GPUs per replica).

Key insight: With EP enabled, `EP_SIZE = TP × DP` determines how many GPUs participate in distributing experts. Whole experts are distributed across the EP grid. Attention layer behavior depends on TP: replicated when TP=1, sharded when TP>1. The router/gating network is always replicated (cheap computation, runs independently on each GPU to compute routing decisions before AllToAll). For a detailed comparison of TP vs DP+EP trade-offs for MoE models, see the AMD ROCm MoE Playbook.

Example topologies:

- **Single-node dense model**: TP=8, DP=1 (8× GPUs in one node)
- **MoE model (EP only)**: TP=1, DP=8, EP enabled (8× GPU, experts distributed across all 8, attention replicated)
- **MoE model (EP + TP)**: TP=2, DP=4, EP enabled (8× GPU, 4 TP groups of 2 GPUs, experts distributed across all 8, attention sharded within TP groups)
- **Massive MoE**: TP=1, DP=16, EP enabled (16× GPUs across 2 nodes, experts distributed across all 16, attention replicated)

KV Cache Management

The KV cache stores attention key and value tensors for each token processed during inference. As context length grows, KV cache becomes the **dominant variable memory consumer** — often exceeding model weight size. After model weights and activation buffers are allocated (see Engine Tuning: `gpu_memory_utilization`), the remainder of GPU memory becomes the **KV cache pool**. How this pool is managed determines how many concurrent requests an endpoint can serve, how efficiently repeated prefixes are handled, and whether long-context workloads fit in memory at all.

This section covers three KV cache optimization strategies:

1. **Prefix caching** — Reuse KV blocks for repeated prompt prefixes
2. **Tiered caching** — Extend cache capacity beyond GPU memory using CPU/disk/remote storage
3. **Cache-aware routing** — Direct requests to instances that already hold relevant KV blocks

Prefix Caching

Many LLM workloads share prompt prefixes across requests. System prompts, few-shot examples, retrieved documents in RAG pipelines, and conversation histories create repeated prefixes that would otherwise be recomputed for every request.

Prefix caching retains KV cache blocks after a request completes and reuses them for subsequent requests with matching prefixes:

1. When a request completes, its KV cache blocks remain in GPU memory instead of being freed
2. New requests are matched against cached prefixes via hash-based lookup
3. If a prefix match is found, the engine skips **prefill** for the matched portion and loads cached KV blocks directly
4. Only the novel suffix (tokens beyond the cached prefix) goes through prefill, which in turn reduces TTFT and improves throughput

When prefix caching helps most:

- RAG pipelines with shared retrieved documents
- Multi-turn chat with accumulating conversation history
- Agentic workflows with repeated system prompts and tool definitions
- Batch evaluation with the same base prompt and different user inputs

Framework configuration:

Framework	Configuration
vLLM	`vllm serve model --enable-prefix-caching`

SGLang	`python -m sglang.launch_server --enable-prefix-caching`

Design consideration: Prefix caching trades GPU memory (cached KV blocks persist) for compute (skip redundant prefill). In memory-constrained scenarios, cached blocks may be evicted to make room for new requests. Eviction policies are typically LRU (least recently used). The next section addresses this limitation.

Tiered KV Cache (Offloading Beyond Local GPU Memory)

GPU memory is finite. Once the KV cache pool is full, the system must evict cached prefixes (reduced cache hit rate). **Tiered caching** extends effective cache capacity by offloading KV blocks elsewhere, such as slower, larger storage tiers:

How tiered caching works:

- **Automatic tiering**: Hot blocks (frequently accessed) stay in GPU memory; warm blocks move to CPU; cold blocks to disk or remote storage based on access patterns
- **Prefetch hot KV blocks**: The system prefetches the next layer's KV blocks from slower tiers, hiding latency
- **Cross-request persistence**: KV blocks survive beyond a single request's lifecycle — cached prefixes remain available even after GPU memory is reclaimed for new requests
- **Cross-worker sharing**: Separate inference engine replicas can share or exchange KV blocks, enabling prefix reuse across instances

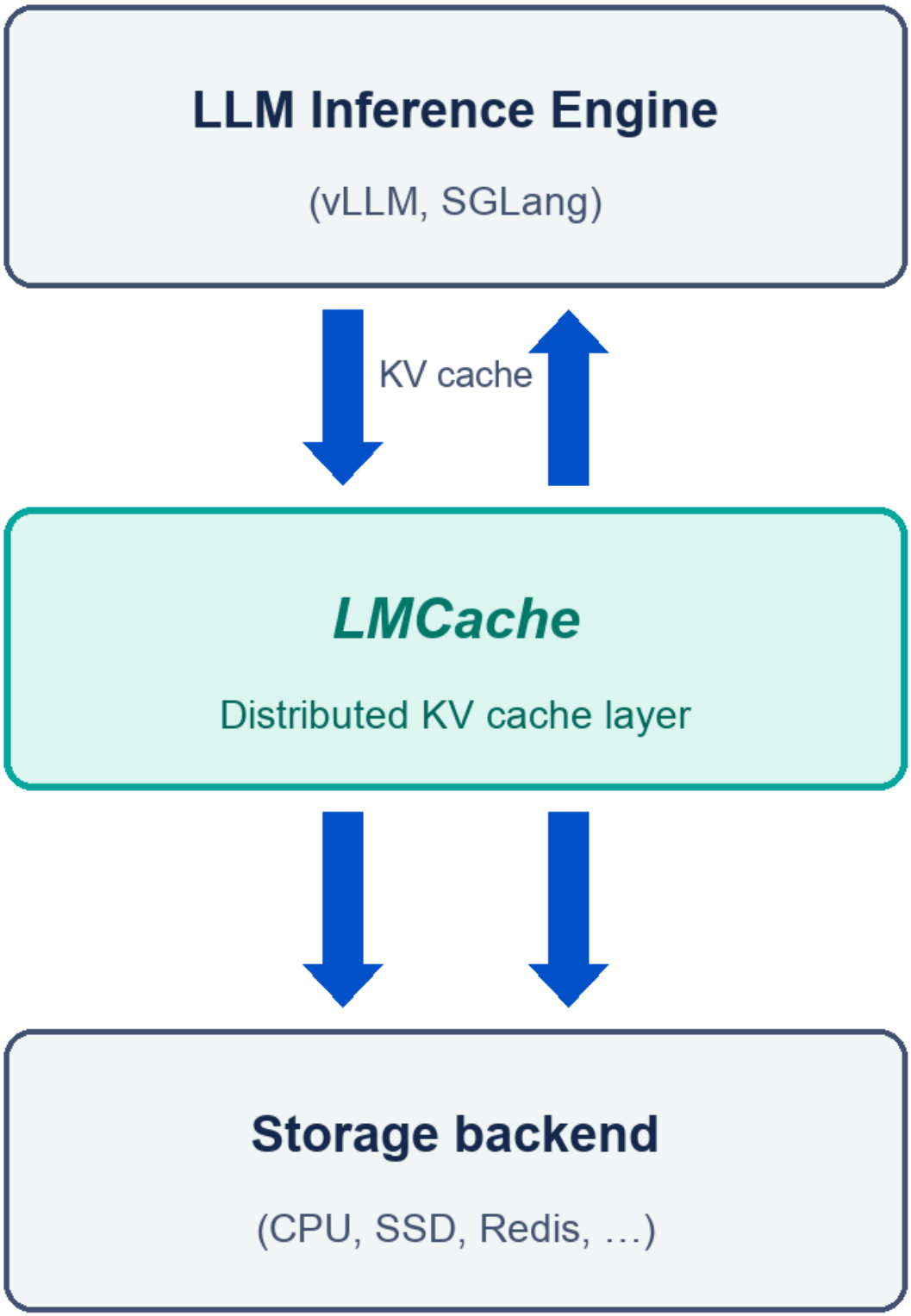
Performance impact:

- **Throughput**: improvement for high-reuse workloads (multi-round Q&A, document analysis)
- **TTFT**: reduction compared to recomputing shared prefixes from scratch
- **Scale**: Enables caching 100K+ unique prefixes across CPU/SSD tiers

Framework integration

Several tools provide tiered caching capabilities. LMCache is one of them, an open-source library that works with vLLM and SGLang, while NVIDIA Dynamo includes KVBM (KV Block Manager) which is tightly integrated with its disaggregation features (covered in the next section). These

capabilities sit between the inference engine and storage tiers, coordinating KV block movement and cross-engine sharing:


LLM Inference Engine
(vLLM, SGLang)
KV cache
LMCache
Distributed KV cache layer
Storage backend
(CPU, SSD, Redis, …)

Deployment example (LMCache with vLLM):

```
vllm serve Qwen/Qwen3-8B \
    --port 8000 --kv-transfer-config \
    '{"kv_connector":"LMCacheMPConnector", "kv_role":"kv_both"}'
```

When to use tiered caching:

- Long-context workloads where GPU memory cannot hold all active KV caches
- Multi-replica deployments where cache sharing across instances improves hit rates
- High-reuse workloads (agentic, multi-turn) where cached prefixes are valuable but expire too quickly from GPU-only storage
- As a building block for prefill-decode disaggregation (see next section)

Design trade-off: Tiered caching adds latency when fetching KV blocks from slower storage (CPU: ~10ms, SSD: ~100ms). I/O pipelining and batched operations reduce this overhead. For workloads with low prefix reuse, the latency cost may outweigh throughput gains.

Cache-aware Request Routing

In multi-replica deployments, naive routing strategies (round-robin, least-loaded, least-requests) ignore KV cache locality. Requests with shared prefixes may land on different replicas, causing cache misses and redundant prefill computation. **Cache-aware routing** directs requests to instances that already hold relevant KV blocks.

How it works:

1. The router maintains metadata about which KV blocks reside on which instances
2. Incoming requests are hashed based on their prefix
3. Requests are routed to instances that already hold their prefix KV blocks, also taking into account GPU load.

Performance impact:

In workloads with high prefix reuse (e.g., same system prompt across thousands of requests), cache-aware routing increases effective cache hit rates translating to TTFT reductions.

Framework support:

Framework/Service	Support
Amazon SageMaker Hyperpod	Managed Tiered KV Cache and Intelligent Routing using HyperPod's Inference Operator
NVIDIA Dynamo	Cache-aware routing (considers KV locality and GPU load in scheduling)
llm-d	Native cache-aware routing via intelligent scheduler
LMCache	Exposes APIs for custom routers to query KV block locations across replicas

Design consideration: Cache-aware routing works best when request distributions are non-uniform. If all requests are unique, routing overhead adds latency without benefit. Monitor cache hit rates before deploying.

Prefill-Decode Disaggregation

LLM inference consists of two phases with opposing resource profiles: **prefill** (compute-bound, parallel processing of the input prompt) and **decode** (memory-bandwidth-bound, sequential token generation). In traditional colocated serving, both phases share the same GPU, causing interference — large prefills block decode steps for other requests, while decode underutilizes compute capacity. **Disaggregation** separates prefill and decode into independent pools, each optimized for its phase's resource profile.

Phase Characteristics

Phase	Bottleneck	Resource need	Duration	Behavior
Prefill	Compute-bound	High FLOPS (tensor cores)	Short burst per request	Parallel, bursty
Decode	Memory-ba ndwidth-bound	High memory bandwidth	Long tail (one token at a time)	Sequential, sustained

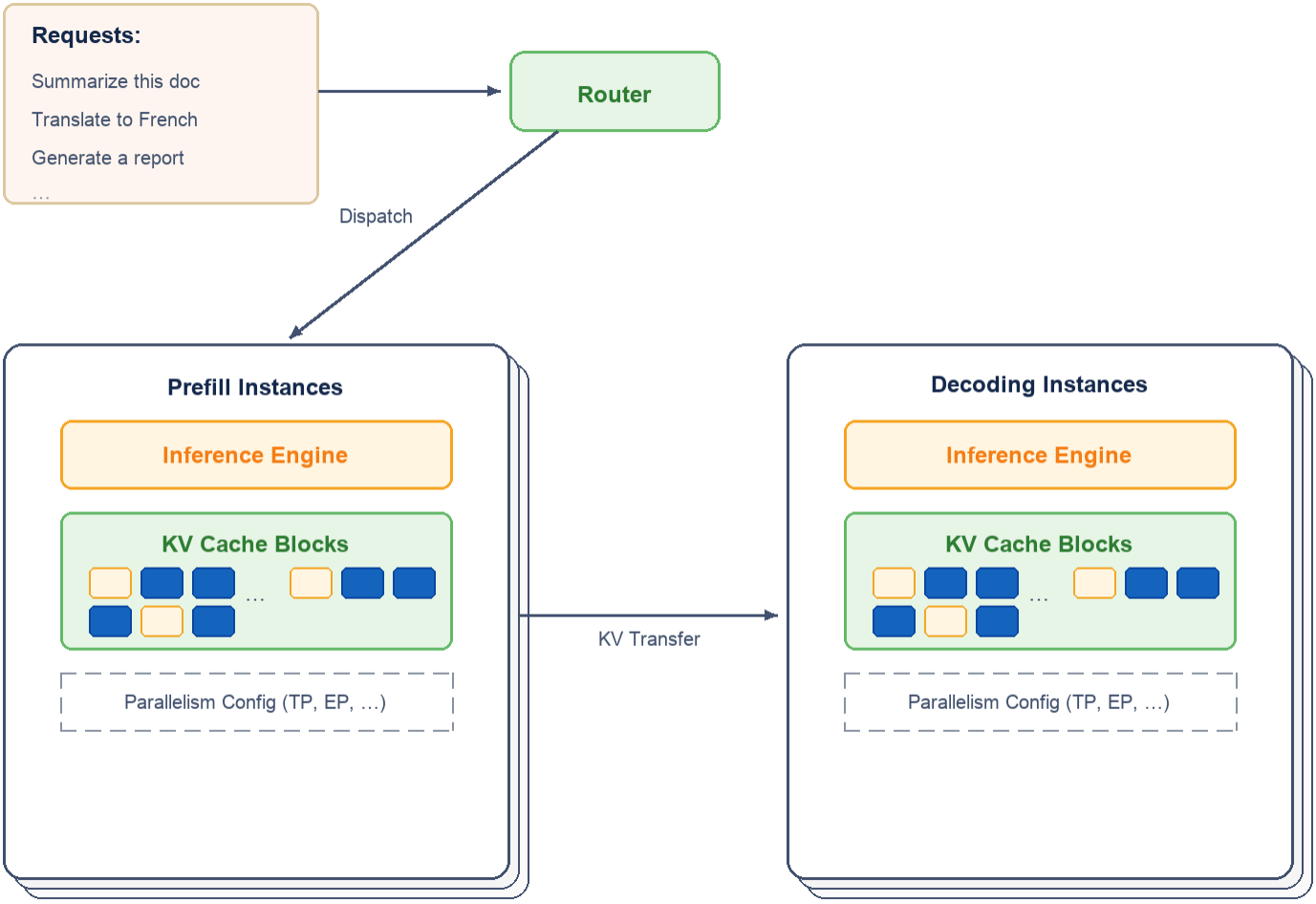
In colocated serving, this mismatch creates decode stalls (prefill blocks other requests), compute underutilization (decode cannot saturate tensor cores), and inability to scale each phase independently.

When to Use Disaggregation

Disaggregation adds architectural complexity (separate pools, KV transfer overhead, routing logic) and typically doubles GPU count. Use this decision framework:

Factor	Use Disaggregation
Model size	Large models (70B+) where prefill causes noticeable decode stalls
I/O ratio	Imbalanced (reasoning/code gen with long outputs, or RAG with long input)
Latency requirements	Cannot tolerate decode latency spikes from concurrent prefills
Concurrency	High concurrency where interference is severe

How Disaggregation Works



Request flow:

- Router directs incoming request to prefill worker (optimized for compute)
- Prefill worker processes full prompt, generating KV cache
- KV cache transfers to decode worker via high-speed interconnect (NVLink/NVSwitch for intra-node, RDMA over AWS EFA for inter-node)
- Decode worker generates output tokens autoregressively using transferred KV cache
- Response streams back to user from decode worker

Encode-Prefill-Decode (EPD) disaggregation for Vision Language Models: VLMs add a compute-intensive **encoding** phase before prefill, where vision encoders process images or video frames into embeddings. EPD (Encoder-Prefill-Decode) disaggregation extends the two-pool architecture to three specialized pools: an encoder pool processes visual inputs, transfers embeddings to the prefill

pool (which combines them with text tokens to generate KV cache), then KV cache transfers to the decode pool for token generation. This isolates vision workload from text processing, enables hardware specialization (vision-optimized accelerators for encoding, LLM-optimized GPUs for text), and allows independent scaling of encoder capacity for vision-heavy workloads.

Performance Impact

Disaggregation delivers several performance benefits. **Independent scaling** allows you to scale prefill and decode pools separately based on workload characteristics — add prefill capacity for long-input workloads or decode capacity for long-output scenarios without overprovisioning the other phase. **Hardware specialization** enables matching hardware to phase requirements: prefill pools use compute-optimized instances with high FLOPS (dense tensor cores), while decode pools use memory-bandwidth-optimized instances with high HBM bandwidth. **Eliminated interference** means decode latency is no longer impacted by concurrent prefill operations, delivering consistent TPOT across all active requests and substantially reducing P99 latency. **Higher utilization** comes from each pool running only the workload it's optimized for, eliminating the underutilization that occurs when a single GPU switches between compute-bound and memory-bound phases.

These gains are most pronounced when prefill and decode have very different resource profiles, such as reasoning models with long generation, long-context RAG with short outputs, or agentic workflows with high token amplification.

KV Cache Transfer: NIXL

Disaggregated serving requires efficient KV cache transfer between prefill and decode pools. **NIXL (NVIDIA Inference Xfer Library)** provides zero-copy, GPU-direct transfers using high-speed interconnects:

- **Intra-node**: NVLink/NVSwitch for GPU-to-GPU transfers within a single node
- **Inter-node**: RDMA over high-bandwidth fabrics (AWS Elastic Fabric Adapter)

NIXL is used by frameworks like NVIDIA Dynamo and llm-d to transfer KV cache blocks without CPU involvement, minimizing latency overhead.

Disaggregated Serving Examples:

- **Amazon Sagemaker HyperPod:** Managed disaggregated serving with vLLM using the HyperPod Inference Operator.

- **NVIDIA Dynamo**: Orchestration layer above vLLM/SGLang with dynamic GPU scheduling and NIXL transport
- **llm-d**: Kubernetes-native framework with NIXL over EFA for AWS deployments (SageMaker HyperPod, EKS)

For detailed deployment guides, see NVIDIA Dynamo documentation and Amazon SageMaker HyperPod.

Right-sizing and auto-scaling an inference system

Fundamental to designing an inference system is the selection and sizing of the underlying compute infrastructure and the policies for dynamically scaling it based on inference demand and inference service-level objectives (SLOs) like end-to-end latency or time to first token (TTFT). We recommend the following steps for right sizing and scaling.

1. Define the Workload

The first step in sizing an inference system is defining the workload requirements. Inference performance is highly dependent on the characteristics of the requests being processed. Two deployments serving the same model can require significantly different infrastructure depending on factors such as prompt length, response length, concurrency, and latency objectives.

Before selecting an accelerator or estimating instance count, gather the following workload characteristics:

Model characteristics

The model architecture and parameter count have a significant impact on both memory requirements and computational throughput. Examples include:

- Llama 3.3 70B
- Mistral Small 24B
- Qwen3 32B
- Gemma 3 27B

In addition to model size, the numerical precision used for inference should also be identified. Common formats include FP16, FP8, INT4, and NVFP4. Lower-precision deployments can reduce memory requirements and improve throughput.

Request characteristics

Inference workloads should be described using both input and output token lengths. Key metrics include:

- Average input tokens per request

- Average output tokens per request
- Peak input tokens
- Peak output tokens

For example, a use case deploying a chatbot might observe requests averaging 500 input tokens and 200 output tokens. A document-processing workload may instead receive prompts containing several thousand input tokens with relatively short responses. Input and output token lengths directly influence latency, throughput, and memory consumption.

Concurrency requirements

The number of requests processed simultaneously is one of the most important inputs for sizing an inference system. Concurrency is typically expressed as:

- Concurrent users
- Concurrent requests
- Requests per second (RPS)
- Requests per minute (RPM)

For example, an application serving 500 active users may only process a small number of concurrent requests if user interactions are infrequent. Conversely, batch-processing workloads can generate sustained high concurrency despite having relatively few users.

Latency objectives

Performance requirements should be defined before infrastructure selection. Common metrics include:

- Time to First Token (TTFT)
- End-to-end response latency (E2E)
- Maximum acceptable queueing delay

Metric	Target
TTFT	< 1 second

E2E	< 5 seconds
Queueing delay	< 500 ms

These objectives determine the amount of infrastructure required and influence the choice of accelerator.

Availability and scaling requirements

Workloads often exhibit varying demand throughout the day. Understanding expected traffic patterns helps determine whether static provisioning or auto scaling is appropriate.

Consider:

- Average utilization
- Peak utilization
- Daily traffic patterns
- Seasonal demand spikes
- Recovery requirements during infrastructure failures

Example workload definition

The following example captures the minimum information required to begin a sizing exercise:

Parameter	Value
Model	Mistral Small 24B
Precision	FP8
Average input length	2,000 tokens
Average output length	500 tokens
TTFT target	< 1 second
End-to-end latency target	< 5 seconds

Once these requirements are established, the next step is determining whether the model and associated KV memory fit within the available accelerator memory and identifying the instance types capable of meeting the required throughput and latency objectives.

2. Memory Sizing: Can the Model Fit?

After the workload requirements have been defined, the next step is determining whether the model can fit within the memory available on a given instance with one or more accelerators.

Memory sizing establishes the set of viable instance size options before throughput and latency considerations are evaluated. If the model and its associated runtime memory requirements exceed the available memory on an instance, that instance cannot be used regardless of its computational performance. Inference memory consumption is primarily driven by three components:

- Model weights and activations
- Key-value (KV) cache
- Runtime overhead

Model weights

The model weights are typically the easiest component to estimate. Memory requirements depend on the model parameter count and the numerical precision used during inference.

The following table shows approximate memory requirements for storing model parameters:

Model Size	FP16	FP8 / INT8	INT4 / NVFP4
7B	14 GB	7 GB	3.5 GB
13B	26 GB	13 GB	6.5 GB
70B	140 GB	70 GB	35 GB

As an example, a 70B parameter model deployed in FP8 requires approximately 70 GB of memory for the model weights alone, exceeding the memory available on a single L40s GPU with 48 GB of High Bandwidth Memory (HBM). Thus, we either need to shard the model across more than one GPU or choose a GPU with a larger HBM like H100 or B200.

KV cache

For many production deployments, the KV cache becomes a significant — sometimes dominant — contributor to total memory consumption. The KV cache stores the **Key** and **Value** tensors from attention layers for previously processed tokens, allowing autoregressive models to generate responses efficiently without recomputing attention over the entire sequence at each step. KV cache requirements scale approximately linearly with:

- Input context length
- Output length
- Concurrent requests (batch size)
- Number of KV heads and layers
- Head dimension (`hidden_size / num_attention_heads`)
- KV cache precision (e.g., 2 bytes for FP16, 1 byte for FP8)

The total KV cache memory can be calculated as:

```
KV cache = 2 × kv_dtype × num_layers × num_kv_heads × head_dim × context_length ×
 batch_size
```

As a result, two deployments serving the same model may require dramatically different memory footprints. For example, using Mistral-7B (GQA, 8 KV heads, 32 layers, head_dim 128, bf16):

Workload	Context Length	KV Cache (1 request)	KV Cache (4 concurrent)
Interactive chatbot	1,000 tokens	0.12 GB	0.49 GB
Document analysis	16,000 tokens	1.95 GB	7.81 GB

Although the model weights remain identical, the second workload can require several times more memory due to KV cache growth.

Model Sharding

Many models exceed the memory capacity of a single accelerator. Sharding the model using techniques like tensor parallelism distributes model weights and KV cache across multiple GPUs within the same node, allowing larger models to be served without reducing context length or quantization precision.

Examples include:

Model	Typical Deployment
Llama 8B FP16	Single GPU
Mistral Small 24B FP16	Single GPU or multi-GPU
Llama 70B FP16	Multi-GPU
Frontier-scale models	Multi-GPU and multi-node

The primary tradeoff is increased communication overhead between GPUs, which can impact latency and throughput depending on the interconnect technology. For more information about sharding techniques, go to the model optimization section.

Accelerator eligibility

After estimating model weights and KV cache requirements, the total memory footprint can be compared against available instance types.

For example:

AWS Instance Family	NVIDIA Accelerator	Memory
g6	L4	22 GB
g6e	L40S	44 GB
g7e	RTX PRO™ 6000 Blackwell	96 GB
p5	H100	80 GB

p5en	H200	141 GB
p6-b200	B200	180 GB
p6-b300	B300	268 GB

This step establishes the set of viable accelerator options before performance benchmarking begins. A deployment requiring 90 GB of memory immediately eliminates single-GPU g6, g6e and p5 instances as candidates, regardless of their throughput characteristics.

Once viable accelerators have been identified based on memory requirements, the next step is determining whether they can meet the workload's latency and throughput objectives.

3. Latency and Throughput sizing for given service requirements

After identifying the instances capable of hosting the model, the next step is determining whether they can satisfy the workload's latency and throughput objectives. Unlike memory sizing, which determines whether a deployment is possible, throughput sizing determines whether a deployment is practical. A model may fit comfortably on an accelerator while still failing to meet the required Time to First Token (TTFT), response latency, or throughput targets.

As a result, benchmark results should always be interpreted within the context of the workload shape that produced them. As explained in the "How AI Inference works" section, large language model inference consists of two distinct phases:

1. Prefill
2. Decode

Each phase places different demands on the underlying hardware and contributes differently to overall application performance. The most common metrics used during throughput sizing include:

Metric	Description
TTFT	Time to First Token
TPS	Generated tokens per second

End-to-end latency	Total request completion time
Concurrent requests	Requests served simultaneously
Requests per second (RPS)	Sustained throughput at a given workload shape

No single metric is sufficient on its own. For example, an accelerator may deliver excellent aggregate throughput while producing unacceptable TTFT under high concurrency. Similarly, an accelerator may achieve excellent TTFT for individual requests while failing to support the required throughput.

Benchmarking representative workloads

Throughput sizing should always be based on workload shapes that resemble production traffic. When evaluating instance types, benchmark configurations should include:

- Representative input lengths
- Representative output lengths
- Expected concurrency levels
- Production inference backend
- Intended quantization level

For example:

Parameter	Value
Model	Mistral Small 24B
Precision	FP8
Input Tokens per request	2,000
Output Tokens per request	500

The resulting benchmark provides a realistic estimate of the latency and throughput that can be expected in production.

From performance measurements to accelerator selection

Once benchmark results are available, candidate instance types can be compared using workload-specific metrics such as TTFT, TPS, and RPS. This comparison enables selection of the accelerator that satisfies both the latency objectives and throughput requirements established in the workload definition phase. The next section discusses how to use these performance measurements to select the most appropriate accelerator and estimate the number of accelerators required to support production traffic.

4. Instance Selection and Sizing Framework

After confirming that a model fits within the available instance HBM and evaluating throughput requirements, the next step is selecting the accelerator that provides the required performance at the lowest practical cost. The goal is to identify the lowest-cost accelerator that satisfies the application's service-level objectives (SLOs).

Compare cost-performance efficiency

Multiple accelerators frequently satisfy both memory and performance requirements. In these situations, cost becomes the primary decision factor. Rather than selecting the accelerator with the highest benchmark result, compare performance relative to infrastructure cost. Example:

NVIDIA Accelerator	Relative Throughput	Relative Cost
L4	1.0x	1.0x
L40S	2.5x	1.7x
H100	3.5x	3.0x
H200	3.8x	3.5x

Although H200 may deliver the highest absolute performance, L40S may provide the best cost-performance ratio for many medium-sized models.

Estimating accelerator count

Once the instance type has been selected based on memory fit and benchmark performance, the next step is determining how many instances are required to support expected production traffic. Express expected production demand using metrics such as:

- Peak concurrent requests
- Peak requests per second
- Expected user growth
- Total token throughput

Compare the measured throughput of candidate instances against expected workload demand. For example, if a workload requires approximately 3,000 tokens per second during peak periods:

Instance	Throughput	Estimated Count
G6e (L40S)	800 tokens/s	4
P5 (H100)	1,500 tokens/s	2
P5en (H200)	1,650 tokens/s	2

Ideally, production deployments include additional capacity beyond the calculated minimum to accommodate traffic spikes, uneven request distribution, infrastructure failures, and future growth.

Using public benchmarks for initial sizing

Benchmarking every instance configuration is often impractical during early architecture design. Public benchmarks from vendor publications, model providers, and benchmark repositories provide useful starting points for narrowing candidate instances and estimating initial capacity. However, results obtained under different workload shapes, quantization strategies, or serving frameworks are not directly comparable.

Benchmarking tools and datasets

Amazon SageMaker AI supports optimized generative AI inference recommendations, a capability that eliminates manual optimization and benchmarking to deliver optimal inference performance.

Instead of manually testing combinations of GPU instance types, serving containers, parallelism strategies, and optimization techniques, you provide your model and workload requirements, and SageMaker AI returns validated, deployment-ready configurations with real performance metrics.

Other common tools include vLLM benchmark suite, LLMPerf, NVIDIA AIPerf, and custom load-testing frameworks (Locust, JMeter). Use synthetic workloads for repeatability and comparison, or production-derived workloads for realistic performance estimates.

Common benchmarking pitfalls

- Comparing benchmarks with different workload shapes
- Evaluating only single-request performance
- Focusing exclusively on throughput while ignoring TTFT or E2E latency
- Assuming public benchmarks will exactly match your production performance

5. Auto Scaling and Capacity Management

After determining the baseline instance count required to satisfy performance objectives, the final step is designing a strategy for handling variations in workload demand. Auto scaling enables inference capacity to adjust dynamically as traffic increases or decreases. This helps maintain latency objectives during peak demand while reducing infrastructure costs during periods of lower utilization.

Establishing baseline capacity

Before configuring auto scaling, a minimum deployment size should be established.

This baseline capacity should be sufficient to:

- Meet expected steady-state traffic
- Maintain target latency objectives
- Tolerate individual accelerator or instance failures
- Absorb short-term traffic fluctuations

Auto scaling should be viewed as a mechanism for handling changes in demand rather than replacing baseline capacity planning.

Horizontal scaling

Most inference deployments scale horizontally by adding or removing accelerator-backed instances.

Horizontal scaling offers several advantages:

- Increased throughput capacity
- Improved resilience
- Simplified operational management
- Independent scaling of inference replicas

Selecting scaling metrics

The effectiveness of an auto-scaling strategy depends heavily on the metrics used to trigger scaling actions. Traditional infrastructure metrics such as CPU utilization often provide limited visibility into inference workload saturation. Instead, scaling decisions should be driven by metrics that more closely reflect user experience and system capacity.

Common examples include:

- Requests per second (RPS)
- Concurrent requests
- Request queue length
- Pending requests
- Time to First Token (TTFT)
- KV cache utilization
- Token throughput per node

Request queue length

Queue depth is often one of the most useful indicators of insufficient capacity.

When incoming traffic exceeds the system's processing capability, requests begin accumulating in a queue before inference starts.

A growing queue typically indicates:

- Insufficient throughput capacity
- Increasing user-facing latency
- Need for additional accelerators

Queue-based scaling policies often provide a direct relationship between scaling actions and user experience.

Latency-driven scaling

For interactive applications, latency metrics may provide better scaling signals than resource utilization.

Examples include:

- Time to First Token (TTFT)
- End-to-end request latency
- P95 response latency
- P99 response latency

If latency exceeds predefined thresholds, additional capacity can be provisioned before user experience degrades significantly.

KV cache utilization

Even though techniques like KV cache offloading and KV cache aware routing have emerged to maximize cache hits and the KV cache memory space. Inference workloads can still be scaled based on KV cache utilization when it reaches a high threshold. Monitoring KV cache utilization can help identify situations where additional replicas are required to add more HBM.

Scaling response time considerations

Unlike stateless web services, inference systems frequently require significant time to scale. Scale-out operations may involve:

- Instance provisioning

- Container startup
- Model weight loading
- CUDA Graph capture
- KV cache initialization
- Framework startup

Depending on model size and infrastructure configuration, these activities may take several minutes. Scaling policies should therefore be configured to react before capacity becomes exhausted.

Reducing Scale-Out and Cold Start Latency

Auto scaling responsiveness can often be improved by reducing the time required to bring new inference replicas into service. Common techniques include:

- Reducing container image download time through container caching in SageMaker AI
- Accelerate container image pull (for example, SOCI parallel pull and unpack on Amazon EKS and Amazon ECR VPC endpoints)
- Using faster storage options for storing or caching model checkpoints and container images like local NVMe or high-performance file system like Amazon FSx for Lustre
- Leverage model weight transfer across nodes (for example, peer-to-peer model transfer with ModelExpress)
- Using Amazon S3 VPC endpoints and Run:ai Model Streamer to accelerate model download when retrieving model artifacts over the network

Scheduled scaling

Some workloads exhibit predictable demand patterns.

Examples include:

- Business-hour traffic
- Batch processing windows
- Regional usage peaks
- Periodic reporting workloads

In these situations, scheduled scaling can complement reactive scaling policies by proactively increasing capacity before demand rises.

Capacity availability considerations

Auto scaling assumes that additional accelerator capacity can be provisioned when required.

In practice, accelerator availability may vary depending on:

- Region and Availability Zone
- Accelerator type
- Account quotas for EC2 and SageMaker AI
- Purchase options (On-demand, On-Demand Capacity Reservations, Spot, ML Capacity Blocks, SageMaker Flexible Training Plans, Saving Plans)

Workloads with strict availability requirements should consider maintaining additional baseline capacity rather than relying entirely on reactive scaling.

Managed scaling options

AWS provides several approaches for implementing inference auto scaling.

Examples include:

- Amazon SageMaker endpoint auto scaling
- Amazon EKS and SageMaker HyperPod with KEDA and Karpenter
- Amazon EKS Auto Mode
- Amazon ECS managed scaling and ECS Managed Instances

The appropriate option depends on the deployment architecture and operational requirements.

Implementing security and responsible AI

Building generative AI (gen AI) models involves complex technical and business decisions. To speed up experimentation, some responsible AI (RAI) evaluations might be overlooked initially. However, conducting an RAI review *before* moving to inference helps organizations strengthen risk assessment and resource management through a structured evaluation process. This pre-release review serves as the final controlled stage where organizations can thoroughly assess models for potential harm. For example, organizations can assess models for bias amplification, privacy violations, safety risks, or misalignment with organizational values.

The RAI review consists of two phases:

- Before moving to inference
- After inference deployment

Before moving to inference

To establish a foundation for responsible deployment, implement a model registry to track lineage, versions, and artifacts, ensuring traceability throughout the model's lifecycle. The model evaluation report provides a detailed assessment that quantifies robustness, fairness across demographic groups, and potential failure modes. This assessment offers visibility into unexpected behavior and supports informed trade-off discussions. These evaluations guide the creation of comprehensive governance structures, including formal approval workflows, stakeholder sign-offs, and escalation paths that maintain organizational awareness of potential risks and impacts.

To complement these governance mechanisms, implement technical safeguards such as multi-layered defensive controls for input validation, output filtering, and personally identifiable information (PII) sanitization. Finally, practical access controls—including API key authentication, identity-based authorization, and network isolation—protect inference endpoints from unauthorized use, abuse, and resource exhaustion.

Model assessment

During the model assessment phase, the evaluation report helps the organization understand the model's robustness and explainability through quantitative and qualitative analyses. This analysis includes evaluating accuracy metrics, fairness scores, and error distributions, complemented by human-in-the-loop reviews and A/B testing to validate real-world performance. The report should

also provide visibility into unexpected behavior, foster trade-off discussions, and guide the team in incorporating mechanisms for exception management.

Documentation preparation

With the model assessment report, the team can now develop internal documentation to record trade-offs and decisions made during the process. This documentation provides valuable context for future development iterations. When the documentation is revisited, it helps the team understand the limitations and requirements of the previous project timeframe.

A model card should include detailed specifications of the model architecture, training methodology, and hyperparameter configurations, along with performance metrics across diverse datasets and demographic groups. It must also explicitly outline both intended use cases and scenarios where the model should not be applied, establishing clear boundaries for appropriate deployment.

Technical safeguards

Technical safeguards protect the system from potential harm through engineering controls and architectural design choices. Before model deployment, developers must establish multi-layered defensive mechanisms, including the following:

- Input validation to detect and reject adversarial prompts
- Output filtering systems to identify and block harmful, misleading, or biased content
- Thorough sanitization processes for PII

Amazon Bedrock Guardrails provides safeguards that you can configure for your generative AI applications based on your use cases and responsible AI policies.

Governance setup

Before deploying a model, organizations should establish formal deployment approval workflows. These procedures help to ensure that all stakeholders review evaluation reports and have access to shared documentation such as model cards and decision records. This approach fosters organizational awareness of potential risks and impacts associated with deployment. A structured approval process should be completed before release. Governance execution must align with previously documented requirements and include technical controls with clear purposes, such as configuring access permissions and rate limits to prevent resource abuse.

It's essential to control access to inference endpoints to prevent abuse, unauthorized use, and excessive consumption—particularly for public- or customer-facing systems. The following complementary access control methods are commonly used:

- **API key authentication** – API keys provide a simple form of authentication that is natively supported by most popular frameworks and API specifications such as the OpenAI API. Managed platforms such as Amazon Bedrock and open source frameworks such as LiteLLM provide options for issuing API keys to users through the UI and programmatically. API keys are associated with allowed models and usage limits. API keys should be securely loaded from an environment variable or key management service on the server and not be exposed to client-side code. Furthermore, API keys should only be valid short-term and rotated frequently, to prevent abuse in case of accidental leakage.
- **Identity-based authentication** – Identity-based authentication relies on short-lived credentials and signed requests instead of shared secrets like API keys. Within AWS, callers assume AWS Identity and Access Management (IAM) roles. Authorization is enforced through IAM policies that scope allowed models, endpoints, and actions (for example, invoke, batch, and embeddings). In public- and customer-facing applications, end users typically authenticate through OAuth2/OIDC (for example, through Amazon Cognito). Then, identities are mapped to their respective IAM roles.
- **Network isolation** – For company-internal inference endpoints, network isolation can be enforced by only exposing access with internal load balancers within a virtual private cloud (VPC) and by scoping access with security groups and network access control lists. If public ingress is unavoidable, use IP allowlists where possible. Also in that situation, the inference endpoint should be fronted by services such as Amazon CloudFront and AWS WAF. These AWS services prevent distributed denial of service (DDoS) attacks and other common exploits. Network isolation complements API keys and identity-based authentication so that even leaked credentials can't reach an endpoint directly.

In addition to access control, request payloads should always be encrypted in transit by enforcing SSL/TLS.

After inference deployment

A gen AI model deployment guided by RAI practices integrates three key elements to ensure models remain safe, effective, and aligned with organizational values:

- Continuous monitoring provides observability through real-time tracking of performance metrics such as latency, throughput, and error rates. It also observes contextual metadata that enables analysis across dimensions like criticality, business domain, compliance, and resource use.
- When issues arise, incident response mechanisms act as a rapid intervention system. Predefined procedures and clear roles and responsibilities help stakeholders access information and resources quickly to minimize impact.
- Complementing these systems, feedback mechanisms capture user experiences that automated metrics might overlook. These mechanisms provide early warnings for hallucinations, bias, or harmful outputs while incorporating diverse perspectives to better reveal model limitations in real-world contexts.

Continuous monitoring

Continuous monitoring provides real-time visibility into model behavior and system performance across the inference lifecycle. Amazon SageMaker AI enables this observability through components such as Model Monitor for detecting data and concept drift, SageMaker Clarify for bias detection and explanations, and SageMaker Debugger for real-time performance tracking. These tools, combined with tagging model interactions by use case, generate contextual metadata that supports deeper analysis and operational insights. For example, critical versus non-critical workload tags enable prioritization during incident response. Business domain tags support performance comparisons. Compliance tags facilitate automated policy enforcement. Cost center tags improve resource tracking and financial accountability.

Incident response mechanisms

Establishing incident response mechanisms helps to mitigate potential harm and keep deployed gen AI models reliable. Address incidents as quickly as possible to minimize their impact on end users. These systems must also be designed for accessibility, so that response stakeholders and subject owners have immediate access to relevant information and resources.

Feedback collection mechanisms

The nondeterministic nature of gen AI models can still lead to unexpected behavior for your customers. Customer feedback serves as an early warning system, identifying hallucinations, biases, or harmful responses that automated quality metrics may miss. Incorporating feedback from diverse user groups helps organizations better understand model limitations and real-world data distributions. By systematically collecting, categorizing, and analyzing both positive and

negative feedback, organizations can continuously refine their understanding of use cases, user values, and expectations.

AWS Partner Network

The AWS Partner Network (APN) is a global community of partners that leverage programs, expertise, and resources to build, market, and sell customer solutions. AWS AI Competency Partners drive the advancement of services, tools, data strategy, and infrastructure pivotal for implementing generative AI (gen AI) technologies across diverse industries. Their customer success and technical best practices have been validated by gen AI specialists at AWS. Partners in the Generative AI category can help organizations in the following ways:

- **Consulting services** – Partners with deep expertise can assist organizations with the following:
  - Adopt and strategize gen AI.
  - Build and test gen AI applications.
  - Train and customize foundation models.
  - Operate and maintain gen AI applications and models.
  - Protect gen AI workloads.
  - Define responsible AI principles and frameworks.
- **Gen AI applications** – Partners provide tools and solutions that deliver customer value across the full AWS gen AI stack, from foundation models to end user applications.
- **Foundation models and app development** – Partners can help organizations with foundation model training, customization, and the development of gen AI-powered applications.
- **Infrastructure and data** – Partners offer expertise in areas like data strategy, governance, storage, and the underlying infrastructure required to power gen AI workloads on AWS.

By leveraging the capabilities of AWS Generative AI Competency Partners, organizations can accelerate their generative AI initiatives, drive innovation, and transform their businesses.

Resources

AWS service documentation

- Amazon Bedrock
- Amazon Bedrock Custom Model Import
- Amazon Bedrock Guardrails
- Amazon SageMaker AI
- Amazon Elastic Compute Cloud
- Amazon Elastic Container Service
- Amazon Elastic Kubernetes Service
- Best Practices for Running AI/ML Workloads on Amazon EKS
- How inference works in Amazon Bedrock
- Deploy models for inference on Amazon SageMaker AI
- Deploying models on Amazon SageMaker HyperPod

Other AWS resources

- AI on Amazon EKS (AWS GitHub repository)
- Benchmark and optimize endpoint deployment in Amazon SageMaker JumpStart (AWS Blog)
- Load & Serve Models on Amazon EKS
- Managed Tiered KV Cache and Intelligent Routing for Amazon SageMaker HyperPod (AWS Blog)
- Responsible AI
- Amazon SageMaker HyperPod workshop (AWS Workshop)

Other resources

- EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test (arXiv)
- An Introduction to Speculative Decoding for Reducing Latency in AI Inference (Nvidia)
- Overview of quantization techniques (Hugging Face)

Contributors

Contributors

The following individuals contributed to this guide.

Authors:

- Oussama Kandakji, Senior AI/ML Go-To-Market Specialist Solutions Architect, AWS
- Nicolas Jourdan, Senior AI/ML Go-To-Market Specialist Solutions Architect, AWS
- Christian Kamwangala, Senior AI/ML Specialist Solutions Architect, AWS
- Hamza Usmani, Senior AI/ML Go-To-Market Specialist Solutions Architect, AWS
- Marco Sgroi, Principal AI/ML Go-To-Market Specialist, AWS
- Mia Chang, AI/ML Go-To-Market Specialist Solutions Architect, AWS
- Paco Gonzalez, Senior Accelerated Compute Go-To-Market Specialist Solutions Architect, AWS
- Vivek Gangasani, WorldWide Tech Leader, SageMaker Inference, AWS

Reviewers:

- Roy Allela, Senior AI/ML Specialist Solutions Architect, AWS
- Robert Northard, Principal Containers Specialist Solutions Architect, AWS

Document history

The following table describes significant changes to this guide.

Change	Description	Date
Initial publication	—	December 18, 2025
Updates	Includes latest AWS service information	August 18, 2026